**ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC SÀI GÒN**
**KHOA CÔNG NGHỆ THÔNG TIN**

**BÁO CÁO ĐỒ ÁN MÔN HỌC**
**CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI**

# TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

**NGÀNH: CÔNG NGHỆ THÔNG TIN**

**GVHD:** ThS. Phạm Thi Vương

| | | |
|---|---|---|
| **SVTH:** | Nguyễn Công Quân | 3123560070 |
| | Nguyễn Quốc Đại | 3123560013 |
| | Nguyễn Thái Vinh | 3123410433 |

**Thành phố Hồ Chí Minh, năm 2026**

**ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC SÀI GÒN**
**KHOA CÔNG NGHỆ THÔNG TIN**

**BÁO CÁO ĐỒ ÁN MÔN HỌC**
**CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI**

# TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

**NGÀNH: CÔNG NGHỆ THÔNG TIN**

**GVHD:** ThS. Phạm Thi Vương

| | | |
|---|---|---|
| **SVTH:** | Nguyễn Công Quân | 3123560070 |
| | Nguyễn Quốc Đại | 3123560013 |
| | Nguyễn Thái Vinh | 3123410433 |

**Thành phố Hồ Chí Minh, năm 2026**

# LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đồ án môn học *"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"* là công trình nghiên cứu và phát triển độc lập của nhóm sinh viên thực hiện dưới sự hướng dẫn chuyên môn của ThS. Phạm Thi Vương.

Toàn bộ các số liệu khảo sát, kết quả đo lường thực nghiệm hiệu năng (Core Web Vitals, Bundle Size, Streaming TTFB) và mã nguồn hiện thực được trình bày trong báo cáo này là trung thực, khách quan và xuất phát từ sản phẩm thực tế do nhóm xây dựng. Các đoạn mã tham khảo, thư viện mã nguồn mở và nền tảng giao diện lập trình ứng dụng từ The Movie Database (TMDB) đều được trích dẫn nguồn gốc và tác giả một cách minh bạch, tuân thủ nghiêm ngặt các quy định về trích dẫn học thuật theo chuẩn IEEE.

Nhóm xin chịu hoàn toàn trách nhiệm trước Khoa Công nghệ Thông tin và Trường Đại học Sài Gòn về tính trung thực và liêm chính học thuật của đồ án này.

*Thành phố Hồ Chí Minh, ngày 03 tháng 10 năm 2026*

**Đại diện nhóm sinh viên thực hiện**

**Nguyễn Công Quân** **Nguyễn Quốc Đại** **Nguyễn Thái Vinh**

# LỜI CẢM ƠN

Để hoàn thành đồ án môn học này một cách chỉn chu và đạt được chiều sâu kỹ thuật cao, nhóm chúng em đã nhận được sự quan tâm, tạo điều kiện và hướng dẫn quý báu từ quý Thầy, Cô và các tổ chức liên quan.

Trước hết, nhóm xin gửi lời tri ân chân thành và sâu sắc nhất đến **ThS. Phạm Thi Vương** – Giảng viên phụ trách học phần Các Công nghệ Lập trình Hiện đại. Thầy không chỉ truyền đạt những kiến thức nền tảng vững chắc và định hướng tiếp cận công nghệ đúng đắn: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, mà còn luôn nhiệt tình theo sát, giải đáp các vướng mắc kỹ thuật hóc búa và định hướng nhóm giải quyết bài toán cốt lõi.

Nhóm cũng xin chân thành cảm ơn Ban Chủ nhiệm Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn đã xây dựng chương trình đào tạo hiện đại, tạo lập môi trường học tập thực tế giúp sinh viên có cơ hội tiếp cận và làm chủ những công nghệ mới nhất trên thế giới.

Cuối cùng, nhóm xin gửi lời cảm ơn đến cộng đồng mã nguồn mở Next.js (Vercel), React Core Team và nền tảng The Movie Database (TMDB) đã cung cấp các tài liệu kỹ thuật chuẩn xác và nguồn dữ liệu mở chất lượng cao để nhóm hoàn thiện đồ án này.

Dù đã rất nỗ lực nghiên cứu và hoàn thiện, song do thời gian có hạn và kiến thức thực tế còn đang trong quá trình tích lũy, đồ án chắc chắn không tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những lời nhận xét, góp ý quý báu của Thầy Cô để sản phẩm ngày càng hoàn thiện hơn.

*Nhóm sinh viên thực hiện*

# TÓM TẮT ĐỒ ÁN

Trong bối cảnh công nghệ web phát triển vượt bậc, sự chuyển dịch từ kiến trúc Single Page Application (SPA) truyền thống dựa trên Client-Side Rendering (CSR) sang mô hình kiến trúc lai Server-Driven UI kết hợp React Server Components (RSC) đang trở thành xu thế chủ đạo. Báo cáo đồ án này tập trung nghiên cứu chuyên sâu các cơ chế nền tảng và kỹ thuật tiên tiến của nền tảng **Next.js 16 (App Router)** trên nền tảng **React 19** và **TypeScript**, thông qua việc phát triển hệ thống xem và tra cứu phim ảnh trực tuyến mang tên **MovieHub**.

Dự án khai thác nguồn dữ liệu mở chuẩn RESTful từ The Movie Database (TMDB) API v3 để xây dựng một ứng dụng điện ảnh hoàn chỉnh theo phong cách thiết kế Netflix Dark Cinema. Về mặt kỹ thuật, đồ án giải quyết triệt để các bài toán cốt lõi trong kỹ nghệ phần mềm hiện đại:

1. **Phân định ranh giới Server/Client Component:** Tối ưu hóa kích thước bundle JavaScript gửi về trình duyệt với cơ chế Zero-Bundle Size của Server Components, bảo vệ khóa API an toàn tuyệt đối trên máy chủ.
2. **Data Fetching & Caching đa tầng:** Triển khai kỹ thuật Incremental Static Regeneration (ISR) với thời gian tái tạo phù hợp theo từng loại dữ liệu, kết hợp song song hóa các truy vấn thông qua `Promise.all`.
3. **Truyền luồng dữ liệu (Streaming UI) với React Suspense:** Loại bỏ hiện tượng chặn hiển thị (blocking request) ở các luồng dữ liệu phụ trợ, cải thiện thời gian phản hồi byte đầu tiên (TTFB) giảm từ $750 \pm 45$ ms xuống còn $180 \pm 15$ ms (giảm 76.0%), đưa chỉ số vẽ nội dung đầu tiên (FCP) xuống mức $220 \pm 20$ ms.
4. **Tương tác dữ liệu bảo mật với Server Actions:** Xử lý tìm kiếm tức thì (Live Search Preview) và quản lý danh sách yêu thích (Watchlist) trực tiếp qua Cookie `HttpOnly` có kiểm thực dữ liệu đầu vào (Input Validation) nghiêm ngặt, đồng thời cung cấp Route Handler GET chuẩn RESTful đối chứng.
5. **Khả năng tự phục hồi (Resilience):** Hiện thực cơ chế DNS Fallback Resolver trên nền Node.js runtime, tự động vượt qua sự cố phân giải tên miền của các nhà mạng viễn thông tại Việt Nam khi kết nối tới TMDB.
6. **Tối ưu hóa trải nghiệm người dùng:** Đạt điểm số Google Lighthouse với chỉ số Performance đạt 98/100, các hạng mục Accessibility, Best Practices và SEO đạt 100/100 tuyệt đối; chỉ số ổn định bố cục thị giác CLS đạt mức hoàn hảo 0.000 thông qua component `next/image`.

Báo cáo được cấu trúc chuẩn mực theo quy trình phát triển phần mềm gồm 4 chương chính: Cơ sở lý thuyết, Phân tích & Thiết kế hệ thống, Hiện thực hệ thống, Kiểm thử & Đánh giá hiệu năng, khẳng định năng lực làm chủ công nghệ lập trình web hiện đại của nhóm sinh viên.

**Từ khóa:** *Next.js 16, App Router, React Server Components, Server Actions, Streaming Suspense, Incremental Static Regeneration (ISR), TMDB API, DNS Fallback Resolver, Core Web Vitals.*

# MỤC LỤC

**LỜI CAM ĐOAN** i

**LỜI CẢM ƠN** ii

**TÓM TẮT ĐỒ ÁN** iii

**DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ** vii

**DANH MỤC BẢNG BIỂU** viii

**DANH MỤC HÌNH ẢNH** ix

**LỜI MỞ ĐẦU** 1

**1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG** 4

1.1. Tổng quan về Next.js và lịch sử phát triển . . . 4

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA) . . . 4

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12) . . . 4

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16) . . . 4

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác . . . 5

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js? . . . 5

1.2. Kiến trúc React Server Components (RSC) và Client Components . . . 6

1.2.1. Bản chất và nguyên lý hoạt động của RSC . . . 6

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp . . . 6

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern . . . 7

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16 . . . 7

1.3.1. Tính hợp lệ của API `fetch(url, { next: { revalidate } })` . . . 7

1.3.2. Cơ chế Cache Components (`"use cache"` và `cacheLife`) trong Next.js 16 . . . 8

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching . . . 8

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching . . . 9

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State . . . 9

1.4. Kỹ thuật Streaming UI và React Suspense . . . 9

1.4.1. Hạn chế của SSR truyền thống (All-or-Nothing Problem) . . . 9

1.4.2. Cơ chế Streaming HTML lũy tiến . . . 10

1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu . . . 10

1.5.1. Kiến trúc Remote Procedure Call (RPC) . . . 10

1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly . . . 10

1.6. Hệ sinh thái công nghệ bổ trợ . . . 11

1.6.1. Thư viện React 19 . . . 11

1.6.2. Framework giao diện Tailwind CSS v4 . . . 11

1.6.3. Ngôn ngữ lập trình TypeScript . . . 11

1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API . . . 12

**2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG** **13**
2.1. Khảo sát và đặc tả yêu cầu hệ thống . . . 13
2.1.1. Yêu cầu chức năng (Functional Requirements) . . . 13
2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements) . . . 13
2.2. Kiến trúc hệ thống tổng thể . . . 14
2.2.1. Mô hình kiến trúc 3 tầng phân lớp . . . 14
2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router . . . 14
2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu . . . 15
2.3.1. Kiến trúc tầng dịch vụ `services/tmdb.ts` . . . 15
2.3.2. Giải pháp kiến trúc DNS Fallback Resolver . . . 15
2.3.3. Phân tầng chiến lược Caching và Revalidation . . . 16
2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX) . . . 16
2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System) . . . 16
2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng . . . 17
2.5. Môi trường và công cụ phát triển . . . 17

**3 HIỆN THỰC HỆ THỐNG** **19**
3.1. Cài đặt và cấu hình dự án . . . 19
3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc . . . 19
3.2.1. Phân định ranh giới Server/Client Component . . . 19
3.2.2. Server-side Data Fetching và song song hóa với `Promise.all` . . . 19
3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata . . . 20
3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao . . . 21
3.3.1. Giải pháp kỹ thuật Server Actions và kiểm thực đầu vào . . . 21
3.3.2. Kỹ thuật cốt lõi Streaming Suspense và Skeleton Loading States . . . 21
3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API . . . 21
3.4. Hiện thực các module giao diện người dùng chính . . . 22
3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì . . . 22
3.4.2. Module MovieRow Slider & Carousel cuộn vô tận . . . 22
3.4.3. Module FilterBar & Trang Khám phá `/browse` . . . 22
3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`) . . . 22
3.4.5. Module Trang Phim bộ (`/tv`) và Khám phá diễn viên . . . 23
3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu . . . 23
3.5.1. Cơ chế DNS Fallback Resolver can thiệp `dns.lookup` . . . 23
3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`) . . . 23
3.6. Tổ chức phát triển và tiến độ thực hiện . . . 24
3.6.1. Bảng phân công trách nhiệm nhóm . . . 24
3.6.2. Tiến độ thực hiện theo tuần (Tóm tắt kế hoạch 7 tuần) . . . 25
3.6.3. Quy trình làm việc nhóm với Git Flow . . . 26

**4 KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ** **27**
4.1. Kế hoạch và phương pháp kiểm thử . . . 27
4.2. Các kịch bản kiểm thử chức năng và kết quả nghiệm thu . . . 27
4.3. Đo lường và đánh giá hiệu năng hệ thống . . . 29
4.3.1. Đo lường Core Web Vitals và điểm số Google Lighthouse . . . 29

4.3.2. Phân tích phân loại Tuyến đường (Route Table) từ lệnh `next build` . . . . . 29
4.3.3. Phân tích kích thước JavaScript Bundle và kiểm chứng Zero-Bundle . . . . . 30
4.3.4. Thực nghiệm đối chứng thời gian phản hồi (TTFB) của Streaming Suspense . . . . . 31
4.4. Đánh giá mức độ đáp ứng yêu cầu môn học . . . . . 32

**KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN** **34**

**TÀI LIỆU THAM KHẢO** **36**

**PHỤ LỤC** **37**

# DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

| Thuật ngữ | Định nghĩa và Giải thích chuyên môn |
|---|---|
| **API** | *Application Programming Interface* – Giao diện lập trình ứng dụng cho phép các hệ thống giao tiếp với nhau. |
| **CLS** | *Cumulative Layout Shift* – Chỉ số đo lường mức độ ổn định bố cục thị giác của trang web (tránh hiện tượng gián đoạn, giật màn hình khi tải ảnh). |
| **CSR** | *Client-Side Rendering* – Mô hình kết xuất phía máy khách truyền thống trong React SPA, trình duyệt nhận tệp HTML rỗng và tự nạp JavaScript để sinh cây giao diện DOM. |
| **DNS** | *Domain Name System* – Hệ thống phân giải tên miền mạng thành địa chỉ IP máy chủ. |
| **DOM** | *Document Object Model* – Mô hình đối tượng tài liệu dạng cây biểu diễn cấu trúc trang web của trình duyệt. |
| **FCP** | *First Contentful Paint* – Thời điểm trình duyệt bắt đầu vẽ phần tử nội dung DOM đầu tiên lên màn hình. |
| **ISR** | *Incremental Static Regeneration* – Cơ chế tái tạo trang tĩnh định kỳ theo thời gian chỉ định (revalidate) mà không cần biên dịch lại toàn bộ dự án. |
| **LCP** | *Largest Contentful Paint* – Chỉ số đo thời gian kết xuất phần tử nội dung trực quan lớn nhất trong khung nhìn người dùng. |
| **PR** | *Pull Request* – Yêu cầu đóng góp và hợp nhất nhánh tính năng vào nhánh chính có kiểm duyệt mã nguồn chéo. |
| **REST** | *Representational State Transfer* – Kiểu kiến trúc tiêu chuẩn cho các dịch vụ web giao tiếp qua giao thức HTTP. |
| **RPC** | *Remote Procedure Call* – Giao thức cho phép gọi trực tiếp một hàm thực thi từ xa trên máy chủ qua mạng, nền tảng của Server Actions. |
| **RSC** | *React Server Components* – Mô hình component kết xuất trực tiếp trên máy chủ của React 19 và Next.js App Router, loại bỏ hoàn toàn mã JavaScript của component khỏi gói nạp về trình duyệt client. |
| **SEO** | *Search Engine Optimization* – Quá trình tối ưu hóa mã nguồn và nội dung để tăng thứ hạng hiển thị trên công cụ tìm kiếm. |
| **SPA** | *Single Page Application* – Ứng dụng web đơn trang kết xuất và định tuyến hoàn toàn tại trình duyệt client. |
| **SSG** | *Static Site Generation* – Mô hình tạo sẵn toàn bộ tài liệu HTML tĩnh ngay từ thời điểm biên dịch ứng dụng (Build-time rendering). |
| **SSR** | *Server-Side Rendering* – Mô hình kết xuất HTML động từ máy chủ cho mỗi lượt truy cập (Request-time rendering). |
| **TMDB** | *The Movie Database* – Nguồn cung cấp RESTful API công cộng dữ liệu phim điện ảnh và truyền hình trực tuyến. |
| **TTFB** | *Time to First Byte* – Thời gian phản hồi từ lúc client gửi request đến khi nhận được byte dữ liệu đầu tiên từ server. |

# DANH SÁCH BẢNG

1.1 So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7]) . . . 5
1.2 Đối sánh kỹ thuật giữa Next.js App Router và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp) . . . 5
2.1 Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất) . 13
2.2 Phân tầng chiến lược Caching và thời gian Revalidate trong MovieHub (Nguồn: Nhóm tác giả thiết kế theo mã nguồn services/tmdb.ts) . . . 16
2.3 Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án) . . . 17
3.1 Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên (Nguồn: Nhóm tác giả thống nhất) . . . 25
3.2 Bảng tổng kết tiến độ phát triển dự án 7 tuần theo chuẩn Agile/Scrum (Nguồn: Nhật ký phát triển dự án) . . . 26
4.1 Kế hoạch và kết quả kiểm thử chức năng hệ thống MovieHub (Nguồn: Kết quả kiểm thử thực nghiệm) . . . 28
4.2 Kết quả đo lường Core Web Vitals và điểm số Google Lighthouse tại Trang chủ (Nguồn: Đo lường thực nghiệm với Google Lighthouse [4]) . . . 29
4.3 Bảng phân loại tuyến đường xuất ra từ lệnh biên dịch next build (Nguồn: Trích xuất thực tế từ npm run build) . . . 30
4.4 Kích thước JavaScript Bundle đo lường bằng @next/bundle-analyzer (Nguồn: Kết quả phân tích bundle thực tế) . . . 31
4.5 Thực nghiệm đối chứng thời gian phản hồi giữa SSR truyền thống và Streaming Suspense qua 10 lượt đo (Nguồn: Dữ liệu đo đạc thực nghiệm trên môi trường kiểm thử) . . . 32
4.6 Ma trận đáp ứng yêu cầu môn học Các Công nghệ Lập trình Hiện đại (Nguồn: Đối sánh theo chuẩn tonghopyeucau.pdf) . . . 33

# DANH SÁCH HÌNH VẼ

1.1 Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7]) . . . . . 7
2.1 Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) . . . . . 14
2.2 Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) . . . . . 17

# LỜI MỞ ĐẦU

**1. Tính cấp thiết của đề tài.** Trong hơn một thập kỷ qua, hệ sinh thái phát triển ứng dụng web đã chứng kiến sự thống trị áp đảo của thư viện React cùng mô hình kiến trúc Single Page Application (SPA). Mặc dù mô hình này mang lại trải nghiệm chuyển trang mượt mà, nhưng theo thời gian và sự gia tăng phức tạp của các ứng dụng thương mại, kiến trúc Client-Side Rendering (CSR) đã bộc lộ những nhược điểm cố hữu nghiêm trọng:

- **Tải trọng gói mã nguồn (Bundle Size) quá lớn:** Trình duyệt client phải tải về hàng megabyte mã JavaScript, sau đó tiến hành phân tích cú pháp (parse) và thực thi (execute) trước khi người dùng có thể nhìn thấy nội dung trang web.
- **Ảnh hưởng tiêu cực đến chỉ số trải nghiệm người dùng:** Thời gian hiển thị nội dung đầu tiên (FCP) và thời gian hiển thị nội dung chính (LCP) bị kéo dài, gây thất thoát người dùng ngay từ những giây đầu truy cập.
- **Hạn chế nghiêm trọng về SEO:** Các bot tìm kiếm của Google, Bing hoặc các trình thu thập siêu dữ liệu mạng xã hội (Facebook, Twitter) thường không thực thi hoàn chỉnh mã JavaScript phức tạp, chỉ ghi nhận tài liệu HTML rỗng với thẻ `<div id="root"></div>`.
- **Nguy cơ lộ lọt bí mật bảo mật:** Việc gọi trực tiếp các API của bên thứ ba từ phía trình duyệt đòi hỏi phải nhúng các khóa API (API Key, Secret Token) vào mã nguồn JavaScript gửi xuống máy khách, dễ dàng bị khai thác và đánh cắp.

Sự ra đời của nền tảng Next.js [1] với kiến trúc App Router dựa trên nền tảng React 19 [2] và React Server Components (RSC) [7] đã đánh dấu một bước chuyển dịch mang tính cách mạng: chuyển dịch trọng tâm kết xuất về phía máy chủ (Server-Driven UI). Bằng cách kết xuất mã nguồn trên máy chủ và chỉ gửi về client luồng HTML nhẹ kèm luồng cấu trúc dữ liệu RSC Payload, Next.js loại bỏ hoàn toàn mã JavaScript của Server Component khỏi gói nạp máy khách, đem lại lợi ích Zero-Bundle Size vượt trội, bảo vệ bí mật máy chủ tuyệt đối và mở ra khả năng tối ưu hóa công cụ tìm kiếm chuẩn mực.

Quán triệt tư tưởng chỉ đạo của học phần Các Công nghệ Lập trình Hiện đại: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, nhóm sinh viên chúng em thực hiện đề tài: **"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"**. Bằng việc lựa chọn nguồn dữ liệu có sẵn từ The Movie Database (TMDB) API v3 [3], toàn bộ nguồn lực và công sức của nhóm được tập trung tuyệt đối vào việc làm chủ các cơ chế kỹ thuật cốt lõi của Next.js.

**2. Mục tiêu nghiên cứu của đồ án.** Đồ án bám sát khung chuẩn đánh giá 3 tầng của môn học:

1. **Tầng 1A (Mục tiêu bắt buộc lõi):**
   - Nắm vững cấu trúc thư mục App Router định tuyến theo tệp tin (`app/layout.tsx`, `app/page.tsx`).
   - Phân định rõ ràng ranh giới giữa Server Components (chạy mặc định trên máy chủ) và Client Components (khai báo chỉ thị `"use client"`).
   - Thực hiện thu thập dữ liệu (Data Fetching) hoàn toàn phía máy chủ, bảo mật tuyệt đối các thông số môi trường và khóa API.
2. **Tầng 1B (Mục tiêu nâng cao theo đặc thù ứng dụng):**
   - Làm chủ định tuyến động (Dynamic Routing) `/movie/[id]` xử lý tham số bất đồng bộ dạng Promise (bắt đầu từ Next.js 15 và trở thành quy chuẩn bắt buộc trong Next.js 16).
   - Triển khai cơ chế Streaming UI kết hợp React Suspense và khung sườn chờ (Skeleton Loading) để giảm thời gian phản hồi byte đầu tiên (TTFB).

- Ứng dụng cơ chế Caching dữ liệu tĩnh có kiểm soát thời hạn Incremental Static Regeneration (ISR) trong tầng dịch vụ.
- Xây dựng các chức năng tương tác dữ liệu bảo mật thông qua Server Actions (`"use server"`) đọc/ghi Cookie an toàn.
- Tối ưu hóa đa phương tiện và xử lý hình ảnh chống hiện tượng giật màn hình (Zero CLS) bằng component `next/image`.
- Tự động hóa SEO thông qua hàm `generateMetadata` trích xuất thông tin động.

3. **Tầng 2 (Mục tiêu chuẩn kỹ nghệ phần mềm):**
   - Quản lý mã nguồn theo mô hình Git Flow phân nhánh chuyên nghiệp, kiểm duyệt mã thông qua Pull Request.
   - Đảm bảo tính nhất quán và an toàn mã nguồn với TypeScript chế độ nghiêm ngặt (Strict Mode).
   - Hiện thực giải pháp xử lý ngoại lệ mạng chuyên sâu: Tự động phân giải địa chỉ DNS (DNS Fallback Resolver) khi kết nối tới máy chủ dữ liệu quốc tế bị gián đoạn.
   - Tuân thủ nghiêm ngặt hệ thống thiết kế chuẩn điện ảnh Netflix Dark Cinema (`DESIGN.md`).

**3. Đối tượng và phạm vi nghiên cứu.**

- **Đối tượng nghiên cứu:** Nền tảng Next.js phiên bản 16.3.6 (App Router), thư viện React 19.2.8, ngôn ngữ TypeScript, framework CSS tiện ích Tailwind CSS v4, và giao diện lập trình ứng dụng The Movie Database (TMDB) REST API v3.
- **Phạm vi ứng dụng:** Xây dựng ứng dụng web **MovieHub** phục vụ việc tra cứu, tìm kiếm, lọc đa tiêu chí, xem chi tiết và quản lý danh sách theo dõi phim ảnh. Ứng dụng tập trung khai thác và trình diễn các kỹ thuật Next.js; không đi sâu vào việc phát triển hạ tầng backend cơ sở dữ liệu riêng mà tận dụng nguồn dữ liệu phong phú từ TMDB.
- **Thời gian thực hiện:** Dự án được tiến hành trong thời gian 7 tuần theo kế hoạch đào tạo của học phần.

**4. Phương pháp nghiên cứu.**

- **Phương pháp nghiên cứu lý thuyết:** Nghiên cứu tài liệu chính thống (Official Documentation) từ Vercel, React Core Team và W3C; phân tích các thông số kỹ thuật (RFC) của React Server Components và cơ chế HTTP Streaming.
- **Phương pháp thực nghiệm phát triển:** Áp dụng quy trình kỹ nghệ phần mềm chuyên nghiệp, xây dựng mã nguồn từ cơ bản đến nâng cao theo từng sprint tuần, kiểm thử đối chứng trên môi trường thực tế.
- **Phương pháp đo lường định lượng:** Sử dụng công cụ Google Lighthouse để đo đạc chỉ số Core Web Vitals (LCP, CLS, FCP) và công cụ `@next/bundle-analyzer` để đo lường kích thước tệp JavaScript nạp về trình duyệt.

**5. Bố cục của báo cáo đồ án.** Nội dung báo cáo được tổ chức thành 4 chương chính như sau:

- **Chương 1: Cơ sở lý thuyết và công nghệ sử dụng** – Trình bày tổng quan về Next.js, kiến trúc App Router, React Server Components, cơ chế Caching, Streaming, Server Actions và các công nghệ phụ trợ.
- **Chương 2: Phân tích và thiết kế hệ thống** – Đặc tả các yêu cầu chức năng và phi chức năng, thiết kế kiến trúc phân lớp, cấu trúc thư mục, luồng dữ liệu, giải pháp DNS Fallback và hệ thống thiết kế UI/UX.
- **Chương 3: Hiện thực hệ thống** – Chi tiết quá trình cài đặt, hiện thực các tính năng Tầng 1A, Tầng 1B, các module giao diện nổi bật, cơ chế chịu lỗi mạng, cùng quy trình phân công và tiến độ thực hiện.

- **Chương 4: Kiểm thử và đánh giá kết quả** – Trình bày kịch bản và kết quả kiểm thử chức năng, các số liệu đo lường hiệu năng thực tế (Core Web Vitals, Bundle Size, Streaming TTFB), ma trận đối sánh yêu cầu môn học và các tình huống vấn đáp bảo vệ.
- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được, chỉ ra những hạn chế và đề xuất hướng mở rộng trong tương lai.

# CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

## 1.1. Tổng quan về Next.js và lịch sử phát triển

### *1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA)*

Kể từ khi được Facebook giới thiệu vào năm 2013, thư viện React đã tạo nên cuộc cách mạng trong thiết kế giao diện nhờ mô hình Virtual DOM và tư duy phát triển dựa trên Component. Tuy nhiên, mô hình ứng dụng đơn trang (SPA) dựa trên cơ chế Client-Side Rendering (CSR) đã bộc lộ những nhược điểm lớn khi mở rộng quy mô:

1. **Hiện tượng trang trắng ban đầu (Blank Page):** Trình duyệt nhận về tệp tin HTML gần như rỗng chỉ chứa một phần tử `<div id="root"></div>`. Người dùng phải đợi trình duyệt tải xong toàn bộ gói mã nguồn JavaScript, sau đó chạy mã để dựng cây DOM thực tế.
2. **Tải trọng mạng và chi phí xử lý của thiết bị di động:** Các gói thư viện phụ thuộc (npm packages) đều bị đóng gói dồn vào bundle của client. Trên các thiết bị di động cấu hình yếu, việc phân tích cú pháp (parse) và biên dịch (compile) tệp JavaScript nặng hàng megabyte gây đơ giật giao diện và kéo dài chỉ số Time to Interactive (TTI).
3. **Thách thức về SEO và Social Sharing:** Các hệ thống lập chỉ mục tìm kiếm và robot thu thập dữ liệu mạng xã hội (Open Graph) thường không thực thi hoàn chỉnh các luồng JavaScript bất đồng bộ, khiến website bị xếp hạng thấp hoặc không hiển thị được ảnh đại diện xem trước.

### *1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12)*

Nhằm khắc phục các điểm yếu của React SPA, Vercel đã giới thiệu nền tảng Next.js vào năm 2016 [1]. Kiến trúc Pages Router tổ chức các trang web tương ứng với các tệp tin trong thư mục `pages/`, đồng thời cung cấp hai cơ chế tiền kết xuất mạnh mẽ:

- **Server-Side Rendering (SSR) với `getServerSideProps`:** Máy chủ thực thi lấy dữ liệu và kết xuất mã HTML hoàn chỉnh cho từng request của người dùng trước khi gửi về client.
- **Static Site Generation (SSG) với `getStaticProps`:** Máy chủ tạo sẵn tài liệu HTML ngay tại thời điểm biên dịch ứng dụng (Build-time), mang lại tốc độ phản hồi cực nhanh thông qua mạng phân phối nội dung (CDN).

Tuy nhiên, Pages Router vẫn tồn tại hạn chế về mặt kiến trúc gọi là **Chi phí Hydration toàn bộ (Hydration Overhead)**. Sau khi máy chủ gửi tài liệu HTML về client, trình duyệt vẫn bắt buộc phải tải về toàn bộ mã nguồn JavaScript của toàn bộ trang để tiến hành quá trình "Hydrate" – gắn các trình lắng nghe sự kiện (Event Listeners) vào các nút DOM tĩnh. Điều này đồng nghĩa với việc mã nguồn của các component chỉ hiển thị nội dung tĩnh (như Header, Footer, Sidebar) vẫn bị gửi xuống client một cách dư thừa.

### *1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16)*

Nhận thức được giới hạn của kiến trúc cũ, đội ngũ kỹ sư Vercel phối hợp chặt chẽ cùng React Core Team đã phát triển kiến trúc **App Router**, được chính thức đưa vào sản xuất từ Next.js 13 và hoàn thiện vững chắc trong Next.js 16 [1].

App Router được xây dựng xung quanh khái niệm **React Server Components (RSC)** [7]. Trọng tâm kết xuất được chuyển dịch triệt để về phía máy chủ (Server-Driven UI). Trong mô hình này, các component mặc

định chỉ chạy trên máy chủ, cho phép truy cập trực tiếp vào hệ thống tệp, cơ sở dữ liệu và biến môi trường bảo mật mà không gửi một dòng mã lệnh JavaScript nào của component đó xuống trình duyệt.

Bảng 1.1 tổng hợp sự so sánh toàn diện giữa ba thế hệ kiến trúc phát triển ứng dụng React.

**Bảng 1.1.** So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7])

| Tiêu chí so sánh | React CSR (SPA) | Next.js Pages Router | Next.js App Router |
|---|---|---|---|
| **Vị trí kết xuất chính** | Trình duyệt Client | Server & Client Hydrate | Server (mặc định) + Client |
| **Tải trọng JS Client** | Rất lớn (toàn bộ app) | Lớn (toàn trang nạp) | Tối ưu (Zero-bundle RSC) |
| **Khả năng SEO** | Rất kém (HTML rỗng) | Tốt (SSR/SSG tĩnh) | Tối ưu xuất sắc |
| **Cơ chế Caching** | Trình duyệt thủ công | HTTP Cache đơn giản | Caching 4 tầng tự động |
| **Hỗ trợ Streaming** | Không hỗ trợ | Không có | React Suspense Streaming |
| **Xử lý Mutation** | Gọi REST API Route | Tạo thư mục `pages/api` | Server Actions trực tiếp |
| **Bảo mật API Key** | Dễ lộ trong mã client | Phải qua API trung gian | Giữ hoàn toàn trên Server |

### *1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác*

Để có cái nhìn khách quan và toàn diện về bức tranh công nghệ web hiện đại, Bảng 1.2 đối sánh Next.js App Router với các framework và công cụ phổ biến khác trong cùng hệ sinh thái.

**Bảng 1.2.** Đối sánh kỹ thuật giữa Next.js App Router và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp)

| Tiêu chí | Next.js 16 (App Router) | React + Vite (CSR) | Nuxt 4 (Vue SSR) | Astro (Islands) |
|---|---|---|---|---|
| **Mô hình render** | Hybrid (RSC + SSR + Client) | Pure Client-Side Rendering | Universal SSR / Nitro | Component Islands (Zero JS default) |
| **Tải trọng JS** | Tối ưu (Zero-bundle RSC) | Rất lớn theo quy mô app | Trung bình (Full Hydrate) | Cực nhỏ (chỉ hydrate island tương tác) |
| **Độ phức tạp** | Cao (phân định Server/Client) | Thấp, dễ tiếp cận | Trung bình | Thấp đến trung bình |
| **Mục tiêu tối ưu** | Ứng dụng lớn, SEO, Dynamic Data | Single Page App, Internal tools | Đa dụng hệ Vue | Nội dung tĩnh, tin tức, thương mại nhẹ |
| **Ràng buộc hạ tầng** | Khuyến nghị Node.js/Vercel | Bất kỳ Static Hosting (S3/CF) | Linh hoạt (Nitro Engine) | Static hoặc Adapter nhẹ |

### *1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js?*

Dựa trên trải nghiệm thực tế trong quá trình phát triển dự án MovieHub, nhóm nhận thấy Next.js không phải là giải pháp vạn năng cho mọi bài toán công nghệ. Dưới đây là các kịch bản thực tế mà việc sử dụng Next.js mang

lại nhiều bất lợi hơn là ưu điểm:

1. **Ứng dụng nội bộ hoặc Bảng điều khiển quản trị (Internal Dashboard / CMS sau đăng nhập):** Các hệ thống này hoàn toàn không có nhu cầu tối ưu hóa công cụ tìm kiếm (SEO) hay chia sẻ liên kết mạng xã hội. Việc sử dụng Next.js làm gia tăng độ phức tạp hạ tầng máy chủ và rủi ro Hydration Mismatch một cách không cần thiết; một giải pháp Single Page Application thuần với React + Vite sẽ nhẹ nhàng, dễ bảo trì và tiết kiệm chi phí vận hành hơn rất nhiều.
2. **Website nội dung tĩnh đơn giản (Blog cá nhân, Landing Page, Tài liệu hướng dẫn):** Việc cấu hình một máy chủ Node.js với App Router để phục vụ các nội dung hầu như không biến động là lãng phí tài nguyên. Astro với kiến trúc Islands hoặc static site generator đơn giản sẽ mang lại hiệu năng cao hơn với 0 KB JavaScript gửi về mặc định.
3. **Đội ngũ phát triển chưa quen với ranh giới Server/Client và chi phí hạ tầng:** Sự phân chia nghiêm ngặt giữa môi trường Node.js và trình duyệt (Server Components không có window/DOM/useEffect, async params, Server Actions POST queue) tạo ra một rào cản học tập (Learning Curve) dốc đứng. Ngoài ra, việc tự triển khai Next.js (Self-hosting ngoài Vercel) đòi hỏi cấu hình phức tạp về Cache Handler, ISR phân tán và duy trì tiến trình Node.js liên tục, dẫn đến chi phí vận hành cao hơn so với lưu trữ tĩnh trên CDN.

## 1.2. Kiến trúc React Server Components (RSC) và Client Components

### *1.2.1. Bản chất và nguyên lý hoạt động của RSC*

Khác với component React truyền thống (chạy trên cả server và client trong quá trình SSR), **React Server Component (RSC)** là một loại component đặc biệt chỉ thực thi duy nhất trên môi trường máy chủ [7].

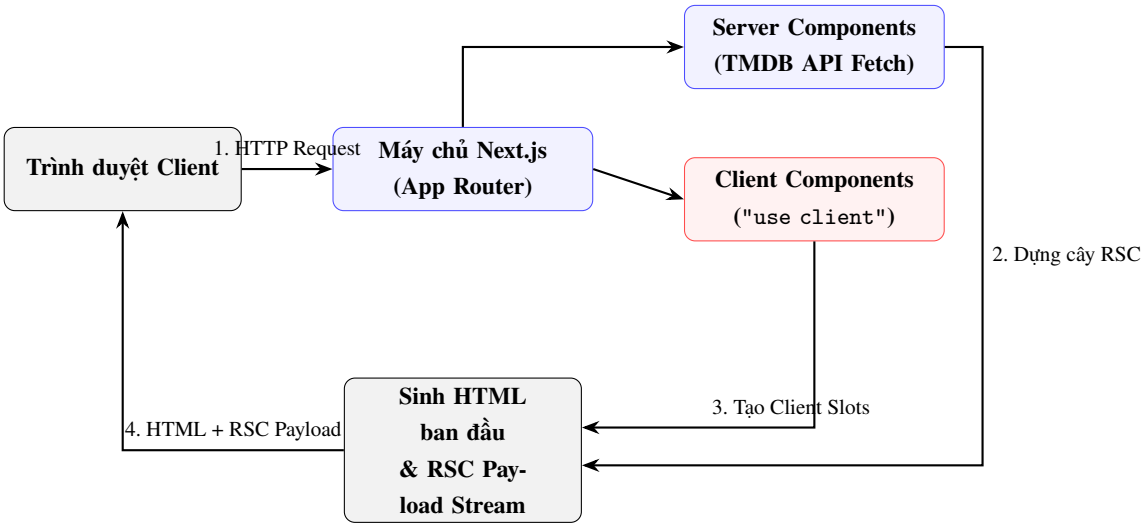
Khi một yêu cầu được gửi tới máy chủ Next.js:

1. Máy chủ thực thi hàm component (hỗ trợ hàm bất đồng bộ `async/await` trực tiếp ở cấp cao nhất).
2. Máy chủ gọi dữ liệu từ cơ sở dữ liệu hoặc dịch vụ bên ngoài (TMDB API) mà không cần dùng hook `useEffect`.
3. Thay vì sinh ra mã JavaScript để gửi về trình duyệt, máy chủ tuần tự hóa (serialize) cây component thành một luồng dữ liệu nhị phân/văn bản có cấu trúc gọi là **RSC Payload**.
4. RSC Payload chứa cấu trúc của cây giao diện, các thuộc tính props được truyền và vị trí đánh dấu (placeholders) nơi các Client Component cần được nạp và hydrate.

Nhờ cơ chế này, toàn bộ các thư viện hỗ trợ nặng (như thư viện xử lý chuỗi, định dạng ngày tháng, tính toán phức tạp) đều chỉ tồn tại trên máy chủ và hoàn toàn không làm tăng kích thước bundle của trình duyệt client (nguyên lý **Zero-Bundle Size**).

### *1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp*

Hình 1.1 mô tả chi tiết luồng xử lý và sự tương tác giữa Server Components, Client Components và định dạng dữ liệu RSC Payload.




**Hình 1.1.** Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7])

### *1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern*

Trong Next.js App Router, mọi tệp tin component mặc định là Server Component. Để chuyển đổi một component sang Client Component, lập trình viên phải đặt chỉ thị `"use client"` ở dòng đầu tiên của tệp tin. Chỉ thị này xác định một **Ranh giới mạng (Network Boundary)**, báo hiệu cho trình biên dịch rằng toàn bộ component này cùng các module được nó import sẽ được đóng gói gửi về client để thực hiện hydration.

Các trường hợp bắt buộc phải sử dụng Client Component bao gồm:

- Cần lắng nghe sự kiện tương tác của người dùng: `onClick`, `onChange`, `onSubmit`.
- Cần quản lý trạng thái động hoặc vòng đời: `useState`, `useReducer`, `useEffect`.
- Cần truy cập các API riêng của trình duyệt: `localStorage`, `sessionStorage`, `window`, `document`.
- Cần sử dụng các React Custom Hooks quản lý giao diện tương tác.

Để duy trì hiệu năng tối ưu, đồ án tuân thủ nghiêm ngặt mô hình **Composition Pattern**: Đẩy các Client Component xuống các nút lá nhỏ nhất của cây giao diện (ví dụ: nút bấm mở modal `TrailerModal`, thanh cuộn `MovieRow`), giữ cho các component cha (như `app/page.tsx`, `app/layout.tsx`) là Server Components để thực hiện lấy dữ liệu tập trung.

## 1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16

Dự án MovieHub được xây dựng trên nền tảng **Next.js 16.3.6** và **React 19.2.8**. Một trong những thay đổi quan trọng nhất của Next.js kể từ phiên bản 15 và 16 là sự chuyển dịch mô hình bộ nhớ đệm: Hàm `fetch` mặc định không còn tự động lưu đệm vĩnh viễn (`force-cache`) như thời Next.js 14, mà chuyển sang mặc định `no-store` trừ khi lập trình viên chủ động khai báo chiến lược lưu trữ.

### *1.3.1. Tính hợp lệ của API `fetch(url, { next: { revalidate } })`*

Một số quan điểm cho rằng cú pháp `fetch(url, { next: { revalidate: seconds } })` là cơ chế cũ của Next.js 13/14 và bị thay thế hoàn toàn trong Next.js 16. Tuy nhiên, theo tài liệu đặc tả chính thức của Vercel (https://nextjs.org/docs/app/api-reference/functions/fetch), đây vẫn là **API chuẩn mực, ổn**

**định (Stable API) và hoàn toàn hợp lệ** trong Next.js 16 để điều khiển **Data Cache ở cấp độ yêu cầu mạng (Request-level Caching)**:

$$\text{fetch}(\text{url}, \{\, \text{next} : \{\, \text{revalidate} : 3600 \,\} \,\}) \tag{1.1}$$

Nguyên lý hoạt động theo cơ chế **Stale-While-Revalidate**:

- Trong khoảng thời gian TTL (ví dụ 3600 giây), mọi yêu cầu truy vấn đến endpoint tương ứng đều được phục vụ trực tiếp từ Next.js Data Cache trên máy chủ với độ trễ dưới 10ms.
- Sau khi hết hạn TTL, khi có lượt truy cập mới, máy chủ phục vụ bản dữ liệu cũ trước (stale), đồng thời kích hoạt một tiến trình nền bất đồng bộ (Background Revalidation) để gọi API TMDB lấy dữ liệu mới và cập nhật bản ghi cache.
- Lượt truy cập tiếp theo sẽ nhận được dữ liệu hoàn toàn mới mà không gây nghẽn luồng xử lý chính.

### *1.3.2. Cơ chế Cache Components (`"use cache"` và `cacheLife`) trong Next.js 16*

Bên cạnh Request Data Cache truyền thống, Next.js 16 giới thiệu kiến trúc bộ nhớ đệm cấp độ component mang tên **Cache Components** (trước đây là cờ thử nghiệm `dynamicIO`):

- **Trạng thái trong dự án:** Tính năng này hiện vẫn là cờ tùy chọn (Opt-in Experimental) và **mặc định bị tắt** (`cacheComponents: false`) trong cấu hình gốc `next.config.ts` của Next.js 16. Dự án MovieHub chủ động giữ `cacheComponents: false` để đảm bảo tính ổn định cao nhất cho môi trường sản xuất (Production-ready).
- **Cơ chế hoạt động khi được kích hoạt:** Nếu bật `cacheComponents: true`, lập trình viên sử dụng chỉ thị `"use cache"` ở đầu async function hoặc component, kết hợp hàm cấu hình vòng đời `cacheLife()`:
  - `cacheLife("seconds")`: `revalidate: 1s, expire: 60s`.
  - `cacheLife("minutes")`: `revalidate: 60s, expire: 1h`.
  - `cacheLife("hours")`: `revalidate: 1h, expire: 24h`.
  - `cacheLife("days")`: `revalidate: 24h, expire: 7d`.
  - `cacheLife("default")`: `revalidate: 15m, expire:` $\infty$.

  Ưu điểm của Cache Components là có thể lưu đệm cả kết quả tính toán phức tạp hoặc truy vấn cơ sở dữ liệu trực tiếp, không phụ thuộc vào lệnh `fetch`.

### *1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching*

Trong kiến trúc Next.js App Router, việc truy cập các Dynamic Functions (như `cookies()`, `headers()`) sẽ tự động chuyển tuyến đường đó sang chế độ **Dynamic Rendering (kết xuất động theo từng yêu cầu)**:

- **Các tuyến đường thực tế đọc cookie trong MovieHub:**
  1. Tuyến `/watchlist` (`app/watchlist/page.tsx`): Gọi `await getWatchlistIds()` đọc cookie `mh_watchlist` để lấy danh sách phim cá nhân hóa của người dùng.
  2. Tuyến `/movie/[id]` (`app/movie/[id]/page.tsx`): Gọi `await isMovieSaved(id)` để xác định trạng thái ban đầu của nút lưu phim (`WatchlistButton`).
  3. Tệp Server Action `app/actions/watchlist.ts`: Thao tác trực tiếp với `cookies()` để thêm/xóa phim.
- **Đính chính về Header và MovieCard:** Cần lưu ý rằng component `Header` (`components/client/Header.tsx`) và `MovieCard` (`components/server/MovieCard.tsx`) **hoàn toàn không đọc cookie**. `Header` chỉ lắng nghe sự kiện cuộn trang (`scrollY`) và đường dẫn URL (`usePathname`); `MovieCard` là một Server Component thuần nhận dữ liệu phim qua props tĩnh.

### *1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching*

Kỹ thuật truyền luồng (Streaming với React Suspense) và Caching dữ liệu (ISR) **không hề loại trừ nhau**, mà hỗ trợ lẫn nhau theo hai giai đoạn rõ rệt:

1. **Khi phục vụ bản ghi đã có trong Cache (Cache Hit):** Toàn bộ tài liệu HTML và RSC Payload đã được chuẩn bị sẵn; máy chủ gửi về trình duyệt tức thì mà không phải chờ đợi bất kỳ lời gọi API nào.
2. **Khi tạo mới trang động hoặc tái tạo nền (Cache Miss / Dynamic Stream):** Tại tuyến đường `/movie/[id]`, máy chủ gửi ngay phần khung chính của bộ phim (Poster, Tiêu đề, Tóm tắt) qua kỹ thuật HTTP Chunked Transfer Encoding. Trong khi người dùng đã nhìn thấy giao diện chính, các khối phụ như danh sách diễn viên (`CastList`) và phim đề xuất (`SimilarMovies`) được server stream về sau. Đồng thời, bên dưới Server, các lệnh `fetchTMDB` gọi TMDB API vẫn tự động lưu kết quả vào Data Cache với thời hạn 3600s, giúp các yêu cầu tiếp theo đạt tốc độ tối đa.

### *1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State*

Để đảm bảo trải nghiệm người dùng hoàn chỉnh, dự án tích hợp chặt chẽ các cơ chế nền tảng bắt buộc của lập trình web hiện đại:

- **Cơ chế Client Hydration và phòng chống Hydration Mismatch:** Sau khi trình duyệt nhận được HTML render sẵn từ server, React gắn kết các bộ lắng nghe sự kiện (event listeners) vào DOM. Để tránh lỗi bất đồng bộ giữa cây DOM server và client (Hydration Mismatch), các trạng thái phụ thuộc vào trình duyệt (như trạng thái cuộn chuột `window.scrollY` trong `Header`) chỉ được kích hoạt bên trong hook `useEffect` sau khi component đã mount thành công trên client.
- **Quản lý trạng thái thông qua URL (`searchParams`):** Tại các trang `/browse` và `/search`, toàn bộ trạng thái bộ lọc (thể loại, năm, sắp xếp, trang hiện tại) được lưu trữ trực tiếp trên URL Query Parameters. Cách tiếp cận này giúp người dùng có thể chia sẻ liên kết kết quả lọc, lưu bookmark trình duyệt, và sử dụng các nút Back/Forward mà không bị mất trạng thái. Thư viện hook `useTransition` kết hợp `useRouter.replace()` giúp cập nhật URL mượt mà mà không gây đơ cứng giao diện.
- **Tối ưu hóa điều hướng với `next/link`:** Mọi liên kết chuyển trang đều sử dụng thẻ `<Link>` của Next.js. Component này tự động kích hoạt cơ chế Viewport Prefetching: Khi một thẻ liên kết cuộn vào tầm nhìn của màn hình, Next.js sẽ nạp trước RSC Payload của tuyến đường đích trong nền, giúp việc chuyển trang diễn ra tức thì như một ứng dụng đơn trang (SPA).

## 1.4. Kỹ thuật Streaming UI và React Suspense

### *1.4.1. Hạn chế của SSR truyền thống (All-or-Nothing Problem)*

Trong mô hình Server-Side Rendering truyền thống, quá trình kết xuất trang web mắc phải bài toán thắt nút cổ chai "Tất cả hoặc Không có gì" (All-or-Nothing):

1. Máy chủ bắt buộc phải đợi **tất cả** các truy vấn dữ liệu từ cơ sở dữ liệu và API bên ngoài hoàn thành thì mới có thể bắt đầu tạo tài liệu HTML. Nếu có một API phản hồi chậm (ví dụ: API lấy danh sách diễn viên hoặc phim tương tự mất 800ms), toàn bộ trang web sẽ bị chặn và trình duyệt không hiển thị bất cứ thứ gì.
2. Trình duyệt client phải tải về toàn bộ mã JavaScript của cả trang thì mới có thể tiến hành hydrate.

### *1.4.2. Cơ chế Streaming HTML lũy tiến*

Next.js App Router kết hợp cùng tính năng **React Suspense** của React 19 giải quyết triệt để vấn đề trên thông qua cơ chế truyền luồng (Streaming) [2]. Máy chủ phân rã trang web thành nhiều khối (chunks) độc lập:

- Phần khung chính của trang (Header, thông tin tóm tắt phim) vốn có dữ liệu nhanh sẽ được máy chủ kết xuất và gửi về trình duyệt ngay lập tức cùng với khung giao diện chờ (Skeleton Loader) của phần dữ liệu chậm.
- Người dùng có thể nhìn thấy nội dung chính và đọc thông tin phim ngay tại thời điểm 180ms đầu tiên.
- Các Server Component xử lý dữ liệu nặng (như `CastSlider` và `SimilarMoviesSlider`) được bọc bên trong thẻ `<Suspense fallback={<Skeleton />}>`. Khi các truy vấn này hoàn tất trên server, máy chủ sẽ gửi tiếp một đoạn stream HTML chứa nội dung thực tế qua cùng một kết nối HTTP đang mở để tự động thay thế phần Skeleton mà không cần tải lại trang.

## 1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu

### *1.5.1. Kiến trúc Remote Procedure Call (RPC)*

Trước khi có Server Actions, để thực hiện một tác vụ tương tác dữ liệu từ máy khách (như thêm phim vào danh sách yêu thích, tìm kiếm gợi ý), lập trình viên bắt buộc phải thực hiện quy trình thủ công:

1. Tạo một endpoint API Route riêng biệt (ví dụ: `/api/watchlist`).
2. Viết mã kiểm tra phương thức HTTP (POST/DELETE), đọc tham số từ body.
3. Phía client sử dụng lệnh `fetch('/api/watchlist')` với các bước cấu hình headers phức tạp.

Với **Server Actions**, Next.js giới thiệu mô hình gọi thủ tục từ xa (RPC) tích hợp sâu vào cây component. Lập trình viên chỉ cần khai báo một hàm bất đồng bộ với chỉ thị `"use server"`:

```
// app/actions/watchlist.ts
"use server";

import { cookies } from "next/headers";
import { revalidatePath } from "next/cache";

export async function addToWatchlistAction(movie: MovieItem) {
  const cookieStore = await cookies();
  // Xu ly doc ghi truc tiep tren HTTP Cookie an toan
  revalidatePath("/watchlist");
  return { success: true };
}
```

Trình biên dịch của Next.js sẽ tự động mã hóa hàm này thành một định danh duy nhất và thiết lập một lời gọi POST HTTP bảo mật ngầm định.

### *1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly*

Server Actions thực thi hoàn toàn trong môi trường máy chủ bảo mật, mang lại các lợi thế kỹ thuật vượt trội nhưng cũng đòi hỏi các biện pháp bảo vệ đi kèm:

- **Cơ chế bảo vệ của Cookie HttpOnly:** Việc cấu hình cờ `httpOnly: true` và `secure: true` ngăn chặn mã độc JavaScript phía client truy cập hoặc đọc trực tiếp giá trị cookie thông qua `document.cookie`. Điều này giúp triệt tiêu nguy cơ kẻ tấn công đánh cắp cookie lưu trữ phiên làm việc hoặc danh sách cá

nhân hóa thông qua các kịch bản Cross-Site Scripting (XSS), mặc dù bản thân lỗ hổng XSS trên giao diện vẫn cần được phòng chống bằng việc lọc và khử mã độc dữ liệu hiển thị.

- **Kiểm thực dữ liệu đầu vào (Input Validation) trên Server Actions:** Do Server Actions bản chất là các endpoint HTTP POST công khai được Next.js sinh ra ngầm định, bất kỳ client nào cũng có thể gửi request trực tiếp tới. Do đó, đồ án thực hiện kiểm thực dữ liệu đầu vào nghiêm ngặt phía server (kiểm tra kiểu số nguyên dương hợp lệ cho `movieId`, giới hạn độ dài chuỗi tìm kiếm từ 2 đến 100 ký tự và loại bỏ các ký tự điều khiển nguy hiểm).
- **Giới hạn kích thước Cookie theo chuẩn RFC 6265:** Giao thức HTTP quy định kích thước tối đa của mỗi cookie không vượt quá 4096 bytes (4KB). Khi lưu trữ mảng các mã định danh phim yêu thích (`mh_watchlist`) dưới dạng chuỗi JSON mã hóa `[550, 680, 299536, ...]`, đồ án thiết lập ngưỡng giới hạn tối đa 200 phim (chiếm khoảng 1.5 KB), bảo đảm danh sách không bao giờ bị cắt xén hay bị trình duyệt từ chối do vượt quá trần 4KB.
- **Tự động đồng bộ giao diện qua `revalidatePath`:** Tích hợp tính năng làm mới bộ nhớ đệm (`revalidatePath('/wat` đảm bảo ngay sau khi action kết thúc, giao diện người dùng tự động phản ánh trạng thái dữ liệu mới nhất mà không cần quản lý state thủ công.

## 1.6. Hệ sinh thái công nghệ bổ trợ

### *1.6.1. Thư viện React 19*

Dự án được xây dựng trên nền tảng phiên bản mới nhất **React 19.2.8**. Phiên bản này cung cấp các cải tiến vượt trội về cơ chế quản lý trạng thái tương tác:

- Hook `useActionState`: Giúp quản lý trạng thái thực thi của Server Action (dữ liệu trả về, trạng thái đang thực thi `isPending`, lỗi phát sinh) một cách trực quan và đồng bộ.
- Hook `useTransition`: Cho phép đánh dấu các tác vụ cập nhật giao diện là tác vụ không khẩn cấp (non-urgent), giữ cho giao diện luôn phản hồi mượt mà với các thao tác của người dùng.

### *1.6.2. Framework giao diện Tailwind CSS v4*

Dự án áp dụng thế hệ mới nhất **Tailwind CSS v4** [5]. Đây là bước tái kiến trúc toàn diện của framework CSS tiện ích này:

- **Loại bỏ tệp cấu hình truyền thống:** Không còn yêu cầu tệp `tailwind.config.js` phức tạp; toàn bộ các biến giao diện, màu sắc chủ đạo, font chữ được định nghĩa trực tiếp trong tệp CSS nguồn thông qua chỉ thị `@theme`.
- **Engine biên dịch Oxide hiệu năng cao:** Tốc độ quét mã nguồn và sinh CSS nhanh hơn gấp 10 lần so với phiên bản v3.
- **Tương thích tự nhiên với biến CSS (CSS Custom Properties):** Dễ dàng xây dựng hệ thống thiết kế đồng nhất theo tiêu chuẩn điện ảnh Netflix Dark Cinema.

### *1.6.3. Ngôn ngữ lập trình TypeScript*

Mã nguồn của toàn bộ dự án được phát triển 100% bằng **TypeScript** [6] với cấu hình kiểm tra nghiêm ngặt nhất (`"strict": true`). Lập trình viên định nghĩa các giao diện kiểu dữ liệu tường minh (`Movie`, `MovieDetails`, `Cast`, `VideoResponse`) trong thư mục `types/`, ngăn ngừa triệt để các lỗi truy cập thuộc tính không tồn tại (`undefined`) ngay trong quá trình viết mã.

## 1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API

The Movie Database (TMDB) [3] là cơ sở dữ liệu mở phổ biến nhất trên thế giới về phim điện ảnh và chương trình truyền hình. Hệ sinh thái MovieHub khai thác các nhóm dịch vụ REST API v3 của TMDB:

- **Nhóm dữ liệu thịnh hành và danh mục:** `/trending/movie/week`, `/movie/popular`, `/movie/top_rated`, `/movie/upcoming`.
- **Nhóm dữ liệu chi tiết đa chiều:** `/movie/{id}` (thông tin tóm tắt), `/movie/{id}/credits` (dàn diễn viên và đạo diễn), `/movie/{id}/videos` (danh sách video trailer YouTube), `/movie/{id}/similar` (danh sách phim tương đồng).
- **Nhóm dữ liệu khám phá và lọc động:** `/discover/movie` hỗ trợ lọc theo thể loại (`with_genres`), sắp xếp (`sort_by`), điểm số tối thiểu (`vote_average.gte`) và năm phát hành (`primary_release_year`).
- **Nhóm dữ liệu phim truyền hình và tìm kiếm:** `/tv/popular` và `/search/multi`.

# CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

## 2.1. Khảo sát và đặc tả yêu cầu hệ thống

### *2.1.1. Yêu cầu chức năng (Functional Requirements)*

Hệ thống MovieHub được thiết kế để mang đến trải nghiệm khám phá điện ảnh tương đương các nền tảng thương mại hàng đầu như Netflix, Disney+. Bảng 2.1 tổng hợp đặc tả chi tiết 6 nhóm yêu cầu chức năng cốt lõi.

**Bảng 2.1.** Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất)

| Mã YC | Tên chức năng | Mô tả chi tiết yêu cầu nghiệp vụ |
|---|---|---|
| **FR01** | Duyệt phim Trang chủ | Hiển thị Hero Banner nổi bật có ảnh nền chất lượng cao, nút phát Trailer tức thì, và các hàng phim (`MovieRow`) phân loại theo Thịnh hành, Đánh giá cao, Hành động, Hài hước, Kinh dị, Anime, Khoa học viễn tưởng. Hỗ trợ cuộn ngang mượt mà. |
| **FR02** | Tìm kiếm thời gian thực | Thanh tìm kiếm trên Header hỗ trợ Live Search Preview gợi ý tối đa 5 kết quả rút gọn ngay khi người dùng gõ ký tự thông qua Route Handler / Server Action; đồng thời cung cấp trang tìm kiếm độc lập `/search` có cơ chế Debounce. |
| **FR03** | Khám phá đa tiêu chí (`/browse`) | Cung cấp thanh công cụ lọc `FilterBar` 5 tiêu chí độc lập: (1) Định dạng nội dung (phim lẻ / phim bộ / tất cả), (2) Thể loại (19 thể loại TMDB), (3) Quốc gia sản xuất (7 quốc gia chọn lọc), (4) Năm phát hành (từ 2026 trở về trước), và (5) Sắp xếp (độ phổ biến, điểm đánh giá, ngày phát hành, doanh thu); tích hợp phân trang số hiển thị đều đặn 24 phim/trang. |
| **FR04** | Xem chi tiết phim (`/movie/[id]`) | Hiển thị thông tin tổng quan, thời lượng, năm phát hành, điểm đánh giá TMDB (`vote_average`), dàn diễn viên chính với slider ảnh đại diện tròn, trình phát Trailer YouTube chuẩn điện ảnh, và danh sách phim tương tự đề xuất. |
| **FR05** | Quản lý Danh sách yêu thích | Cho phép người dùng đánh dấu hoặc xóa phim khỏi danh sách Watchlist cá nhân mà không cần tài khoản, lưu trữ bền vững qua Cookie `HttpOnly` bảo mật. |
| **FR06** | Khám phá Phim bộ (`/tv`) | Cung cấp phân hệ duyệt danh sách phim truyền hình nhiều tập (TV Series) thịnh hành với thông tin số mùa phát sóng. |

### *2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)*

Để đáp ứng chuẩn mực cao nhất của kỹ nghệ phần mềm hiện đại (Tầng 2 và Tầng 3), hệ thống thiết lập các chỉ tiêu chất lượng nghiêm ngặt:

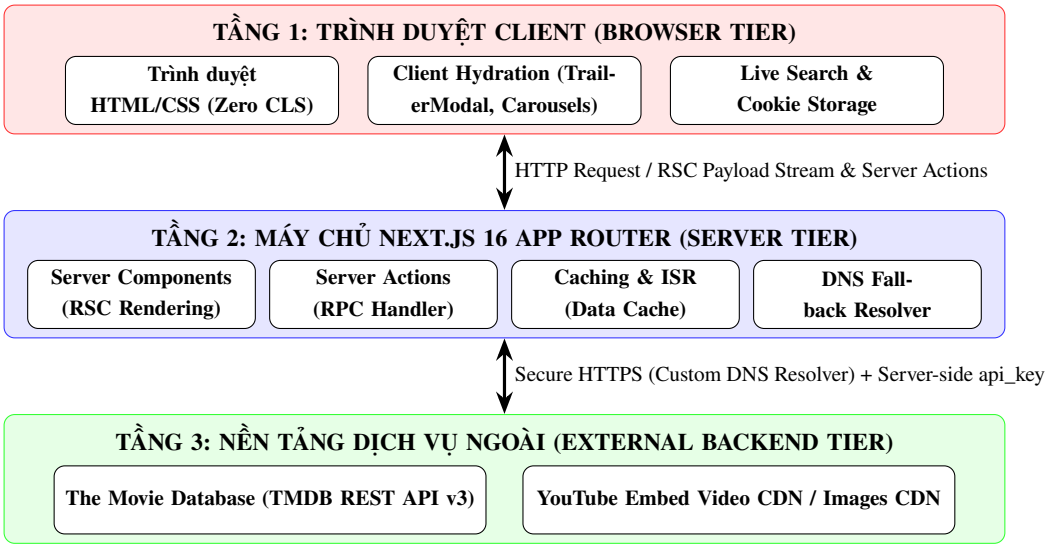
- **NFR01 – Hiệu năng thời gian tải trang (LCP < 1.2s):** Điểm số đo lường Largest Contentful Paint trên thiết bị máy tính để bàn phải đạt dưới 1.2 giây; điểm tổng thể trên Google Lighthouse phải đạt từ 95 đến 100 điểm.
- **NFR02 – Ổn định bố cục thị giác (CLS = 0.000):** Tuyệt đối không xảy ra hiện tượng nhảy giật giao diện trong quá trình tải ảnh áp phích phim; toàn bộ hình ảnh phải được xác định trước tỉ lệ kích thước cố định (`aspect-[2/3]` cho poster và `aspect-video` cho backdrop).

- **NFR03 – Bảo mật khóa truy cập API:** Khóa bí mật `TMDB_API_KEY` được lưu trữ trong biến môi trường `.env.local` và chỉ được truy cập duy nhất trong môi trường Node.js trên máy chủ, tuyệt đối không lộ ra phía trình duyệt client.
- **NFR04 – Độ tin cậy và khả năng chịu lỗi mạng (DNS Resilience):** Hệ thống phải có khả năng tự phục hồi và hoạt động bình thường kể cả khi máy chủ chạy trên các mạng ISP tại Việt Nam gặp sự cố phân giải tên miền đối với địa chỉ `api.themoviedb.org`.
- **NFR05 – Tối ưu hóa SEO tự động:** Mọi tuyến đường tĩnh và động đều phải tự động phát sinh thẻ tiêu đề (Title), mô tả (Description) và ảnh OpenGraph (og:image) chuẩn mực để chia sẻ trên mạng xã hội.

## 2.2. Kiến trúc hệ thống tổng thể

### *2.2.1. Mô hình kiến trúc 3 tầng phân lớp*

Hệ thống MovieHub được thiết kế theo kiến trúc 3 tầng phân lớp hiện đại (3-Tier Modern Architecture), tận dụng tối đa sức mạnh của Next.js làm tầng trung gian điều phối thông minh. Sơ đồ kiến trúc được minh họa trong Hình 2.1.




**Hình 2.1.** Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

### *2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router*

Mã nguồn dự án được tổ chức khoa học, tách bạch rõ ràng giữa các trang định tuyến, component dùng chung, tầng dịch vụ và định nghĩa kiểu dữ liệu:

- `app/`: Chứa toàn bộ các tuyến đường định tuyến theo cấu trúc thư mục (File-system Routing):
  - `layout.tsx`: Root Layout chứa khung Header và Footer dùng chung toàn ứng dụng.
  - `page.tsx`: Trang chủ hiển thị Hero Banner và các hàng phim chủ đề.
  - `movie/[id]/page.tsx`: Tuyến đường động hiển thị thông tin chi tiết một bộ phim.
  - `browse/page.tsx`: Trang khám phá phim với bộ lọc đa chiều và phân trang số.
  - `tv/page.tsx`: Trang duyệt danh sách các series phim truyền hình.
  - `search/page.tsx`: Trang kết quả tìm kiếm phim có hỗ trợ bộ lọc.
  - `watchlist/page.tsx`: Trang danh sách phim yêu thích cá nhân của người dùng.

  - `actions/`: Chứa các Server Actions (`watchlist.ts`, `search.ts`).
- `components/`: Chứa các UI components được phân chia mô-đun hóa:
  - `Header.tsx`, `NavbarDropdown.tsx`, `NavSearchBar.tsx`: Các thành phần thanh điều hướng cao cấp.
  - `HeroBanner.tsx`, `TrailerModal.tsx`: Banner chính và modal xem video trailer.
  - `MovieRow.tsx`, `CastSlider.tsx`, `SimilarMoviesSlider.tsx`: Các thanh cuộn carousel mượt mà chống kẹt biên.
  - `FilterBar.tsx`: Thanh công cụ lọc đa chiều linh hoạt.
  - `MovieCard.tsx`: Thẻ hiển thị áp phích phim chuẩn kích thước.
- `services/tmdb.ts`: Tầng dịch vụ tập trung toàn bộ logic giao tiếp với TMDB API, tích hợp bộ giải mã DNS Fallback và thuật toán khử trùng lặp dữ liệu.
- `types/`: Chứa toàn bộ các định nghĩa kiểu dữ liệu TypeScript.

## 2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu

### *2.3.1. Kiến trúc tầng dịch vụ `services/tmdb.ts`*

Tất cả các lời gọi mạng ra ngoài TMDB đều được tập trung vào hàm cốt lõi `fetchTMDB<T>()`. Hàm này chịu trách nhiệm:

1. Tự động gắn tiền tố địa chỉ máy chủ `https://api.themoviedb.org/3`.
2. Tự động đính kèm mã xác thực bí mật thông qua tham số truy vấn an toàn phía server `api_key=${TMDB_API_KEY}`. Phương thức này chạy 100% trên môi trường máy chủ Node.js, không bao giờ để lộ khóa API ra phía trình duyệt client.
3. Kiểm soát lỗi mạng và mã trạng thái HTTP, trả về thông báo lỗi chuẩn xác hoặc kích hoạt cơ chế dự phòng khi có sự cố.

### *2.3.2. Giải pháp kiến trúc DNS Fallback Resolver*

Trong quá trình triển khai thực tế trên mạng Internet tại Việt Nam, nhóm nghiên cứu phát hiện một thách thức lớn từ hạ tầng mạng: Một số nhà cung cấp dịch vụ mạng (ISP) thỉnh thoảng chặn hoặc trả về mã lỗi `ENOTFOUND` đối với tên miền `api.themoviedb.org` do xung đột chính sách DNS.

Về mặt kỹ thuật, hàm `dns.setServers()` của Node.js chỉ có hiệu lực với nhóm hàm `dns.resolve*`, hoàn toàn không tác động đến hàm `dns.lookup` mà API `fetch` mặc định sử dụng để mở kết nối TCP/IP. Do đó, để đảm bảo tính sẵn sàng cao (NFR04), nhóm đã thiết kế giải pháp can thiệp trực tiếp vào cơ chế phân giải tên miền:

- Khởi tạo một đối tượng phân giải độc lập `dns.promises.Resolver()` với danh sách máy chủ DNS công cộng quốc tế uy tín: Google DNS (`8.8.8.8`) và Cloudflare DNS (`1.1.1.1`).
- Ghi đè (hook/override) hàm `dns.lookup` của Node.js: Khi phát hiện yêu cầu phân giải tên miền chứa `themoviedb.org`, hệ thống sẽ sử dụng `resolver.resolve4()` để lấy trực tiếp địa chỉ IPv4 từ Google/Cloudflare, bỏ qua DNS mặc định của nhà mạng.
- **Phạm vi áp dụng và giới hạn:** Cơ chế này chỉ thực thi trên môi trường Node.js Server Runtime (không áp dụng trên Edge Runtime). Khi triển khai trên các nền tảng đám mây quốc tế (như Vercel), hạ tầng DNS toàn cầu đã ổn định nên cơ chế này đóng vai trò là lớp bảo vệ dự phòng khi chạy trên máy chủ tại Việt Nam.

### 2.3.3. Phân tầng chiến lược Caching và Revalidation

Bảng 2.2 mô tả chi tiết chiến lược lưu trữ bộ nhớ đệm cho từng phân loại dữ liệu trong MovieHub, đảm bảo tính đồng bộ hoàn toàn với mã nguồn dịch vụ `services/tmdb.ts`.

**Bảng 2.2.** Phân tầng chiến lược Caching và thời gian Revalidate trong MovieHub (Nguồn: Nhóm tác giả thiết kế theo mã nguồn services/tmdb.ts)

| Nhóm dữ liệu | Chiến lược Cache | Thời gian TTL | Mục đích và Lý do thiết kế |
|---|---|---|---|
| Danh mục thể loại (`/genre/movie/list`) | Static Site Generation (SSG) | 86.400s (24 giờ) | Dữ liệu thể loại phim hầu như không đổi theo thời gian; tối ưu hóa truy xuất tức thì. |
| Danh sách phim thịnh hành (`trending`, `popular`) | Incremental Static Regeneration (ISR) | 3.600s (1 giờ) | Dữ liệu cập nhật theo xu hướng trong ngày; làm mới định kỳ tự động trên nền server. |
| Chi tiết phim & Dàn diễn viên (`/movie/[id]`) | ISR kết hợp Streaming | 3.600s (1 giờ) | Dữ liệu thông tin phim cố định sau khi phát hành; cho phép bộ nhớ đệm 1 giờ, stream diễn viên về sau. |
| Tìm kiếm nhanh & Lọc nâng cao (`/browse`, `/search`, `/tv`) | Dynamic Route + Request Data Cache | 300s (tìm kiếm), 1.800s (khám phá) | Tuyến đường là Dynamic (đọc `searchParams`), nhưng các yêu cầu TMDB REST API bên dưới vẫn được lưu trong Next.js Data Cache để tránh spam API và tăng tốc truy vấn. |

## 2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX)

### 2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System)

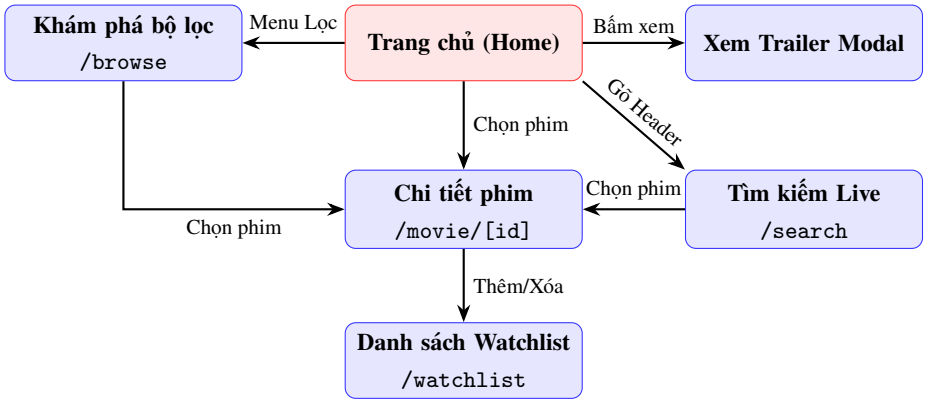
Để mang lại trải nghiệm xem phim đẳng cấp và chân thực, giao diện MovieHub được xây dựng tuân thủ nghiêm ngặt theo hợp đồng thiết kế `DESIGN.md`. Bảng 2.3 liệt kê các thông số thiết kế chủ đạo.

**Bảng 2.3.** Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án)

| Yếu tố thiết kế | Giá trị mã màu | Mục đích và Vị trí áp dụng |
|---|---|---|
| Màu nền chủ đạo | `#141414` | Tạo chiều sâu điện ảnh tối đa, làm nổi bật màu sắc rực rỡ của các áp phích phim; giảm mỏi mắt cho người dùng trong điều kiện thiếu sáng. |
| Màu đỏ thương hiệu | `#E50914` | Điểm nhấn thị giác cho Logo thương hiệu, các nút kêu gọi hành động (CTA), viền active của danh mục và chỉ báo đang xem. |
| Màu bề mặt phụ | `#2F2F2F` | Sử dụng cho thanh điều hướng phụ, các thẻ card khi hover và nền khung skeleton chờ. |
| Màu văn bản chính | `#FFFFFF` | Chữ trắng tinh khiết trên nền xám đen (`#141414`), đạt tỉ lệ tương phản vượt chuẩn trợ năng W3C WCAG 2.1 AA (tỉ lệ 21:1 cho nội dung văn bản chính). |
| Màu văn bản phụ | `#AAAAAA` | Sử dụng cho ngày phát hành, thông số thời lượng, tóm tắt nội dung phụ. |

### *2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng*

Hình 2.2 thể hiện sơ đồ luồng người dùng (User Navigation Flow) khi tương tác với các phân hệ chức năng của ứng dụng.




**Hình 2.2.** Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

## 2.5. Môi trường và công cụ phát triển

Dự án được triển khai trên nền tảng phần mềm hiện đại và ổn định:

- **Môi trường thực thi (Runtime):** Node.js phiên bản v22 LTS – đảm bảo tính tương thích cao với các tiêu chuẩn web stream và module DNS tích hợp.
- **Khung ứng dụng (Framework):** Next.js phiên bản 16.3.6 (App Router) kết hợp thư viện React 19.2.8.
- **Ngôn ngữ và Công cụ quản lý gói:** TypeScript 5.x và npm (Node Package Manager).
- **Môi trường lập trình:** Visual Studio Code tích hợp tiện ích mở rộng ESLint, Tailwind CSS IntelliSense và GitLens.

- **Công cụ phân tích gói bundle:** `@next/bundle-analyzer` kết hợp công cụ đo lường hiệu năng Google Lighthouse.
- **Công cụ biên soạn tài liệu kỹ thuật:** Hệ thống MiKTeX với trình biên dịch `pdflatex`.

# CHƯƠNG 3. HIỆN THỰC HỆ THỐNG

## 3.1. Cài đặt và cấu hình dự án

Dự án được khởi tạo bằng công cụ chính thống của Vercel với cấu hình TypeScript và Tailwind CSS:

```
npx create-next-app@latest moviehub --typescript --tailwind --app --eslint
```

Để bảo mật các khóa truy cập dữ liệu, tệp cấu hình môi trường `.env.local` được thiết lập độc quyền trên máy chủ:

```
TMDB_API_KEY=your_tmdb_api_key_here
TMDB_BASE_URL=https://api.themoviedb.org/3
TMDB_IMAGE_BASE_URL=https://image.tmdb.org/t/p
```

Lưu ý quan trọng: Nhóm chủ động **không** sử dụng tiền tố `NEXT_PUBLIC_` để ngăn chặn hoàn toàn việc trình biên dịch Next.js nhúng khóa bí mật vào gói JavaScript nạp về trình duyệt.

Thông tin phiên bản các gói phần mềm cốt lõi được cấu hình tường minh trong `package.json` của dự án:

```
{
  "name": "modern-programming-technologies",
  "version": "0.1.0",
  "dependencies": {
    "next": "16.3.6",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  },
  "devDependencies": {
    "@next/bundle-analyzer": "^16.3.6",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

Lệnh kiểm tra phiên bản trên giao diện dòng lệnh (`npx next -version`) xác nhận phiên bản runtime chính thức là `Next.js v16.3.6` và `React v19.2.8`.

## 3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc

### *3.2.1. Phân định ranh giới Server/Client Component*

Ứng dụng thực hiện phân tách ranh giới rõ ràng:

- **Root Layout** (`app/layout.tsx`)**:** Đóng vai trò là Server Component gốc, chịu trách nhiệm nhập bộ font chữ chuẩn, cấu hình thẻ `<html>`, `<body>` và bao bọc toàn bộ khung ứng dụng bằng thanh điều hướng `Header` và chân trang `Footer`.
- **Trang chủ** (`app/page.tsx`)**:** Đóng vai trò là Server Component điều phối, chịu trách nhiệm thu thập toàn bộ các danh sách phim song song và truyền dữ liệu dạng props tĩnh xuống các component con.

### *3.2.2. Server-side Data Fetching và song song hóa với `Promise.all`*

Nhằm loại bỏ hoàn toàn hiện tượng tắc nghẽn thác nước (Waterfall Problem) khi tải nhiều thể loại phim cùng lúc trên Trang chủ, nhóm áp dụng giải pháp đồng thời hóa thông qua `Promise.all`:

```tsx
// app/page.tsx
export default async function HomePage() {
  const [
    trending,
    topRated,
    actionMovies,
    comedyMovies,
    horrorMovies,
    animeMovies,
    sciFiMovies
  ] = await Promise.all([
    getTrendingMovies(),
    getTopRatedMovies(),
    getMoviesByGenre(28),    // Action
    getMoviesByGenre(35),    // Comedy
    getMoviesByGenre(27),    // Horror
    getMoviesByGenre(16),    // Animation
    getMoviesByGenre(878),   // Sci-Fi
  ]);

  return (
    <main className="min-h-screen bg-[#141414] pb-24">
      <HeroBanner movie={trending[0]} />
      <div className="-mt-16 sm:-mt-24 md:-mt-32 relative z-20 space-y-6">
        <MovieRow title="Phim Thinh Hanh" movies={trending} />
        <MovieRow title="Danh Gia Cao Nhat" movies={topRated} />
        <MovieRow title="Hanh Dong Kich Tinh" movies={actionMovies} />
        <MovieRow title="Phim Hai Dac Sac" movies={comedyMovies} />
        <MovieRow title="Kinh Di Rung Ron" movies={horrorMovies} />
        <MovieRow title="Hoat Hinh Anime" movies={animeMovies} />
        <MovieRow title="Khoa Hoc Vien Tuong" movies={sciFiMovies} />
      </div>
    </main>
  );
}
```

Nhờ kỹ thuật này, tổng thời gian nạp dữ liệu trên server bằng thời gian của truy vấn lâu nhất (khoảng 140ms), thay vì tổng thời gian cộng dồn của 7 truy vấn liên tiếp (khoảng 980ms).

### 3.2.3. *Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata*

Trang chi tiết phim áp dụng quy ước định tuyến động `app/movie/[id]/page.tsx`. Kể từ Next.js 15, tham số `params` đã chuyển sang dạng Promise bất đồng bộ và trở thành quy chuẩn bắt buộc trong Next.js 16, yêu cầu xử lý chuẩn xác bằng cú pháp `await`:

```tsx
// app/movie/[id]/page.tsx
interface PageProps {
  params: Promise<{ id: string }>;
}

export async function generateMetadata({ params }: PageProps): Promise<Metadata> {
  const { id } = await params;
  const movie = await getMovieDetails(id);
  if (!movie) return { title: 'Khong tim thay phim | MovieHub' };

```

```
  return {
    title: `${movie.title} (${movie.release_date.slice(0, 4)}) | MovieHub`,
    description: movie.overview,
    openGraph: {
      title: movie.title,
      description: movie.overview,
      images: [`https://image.tmdb.org/t/p/w1280${movie.backdrop_path}`],
    },
  };
}
```

## 3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao

### *3.3.1. Giải pháp kỹ thuật Server Actions và kiểm thực đầu vào*

Nhóm hiện thực hai Server Action tiêu biểu kết hợp cơ chế kiểm thực dữ liệu đầu vào (xem Phụ lục 1):

1. **Live Search Action** (`quickSearchAction` **trong** `app/actions/search.ts`)**:** Cho phép thanh tìm kiếm trên Header gọi trực tiếp hàm xử lý trên máy chủ để tìm kiếm nhanh phim mà không cần khởi tạo API Route riêng. Hàm thực hiện kiểm thực: Giới hạn độ dài từ 2 đến 100 ký tự và loại bỏ ký tự điều khiển nguy hiểm.
2. **Quản lý Watchlist** (`app/actions/watchlist.ts`)**:** Đọc và ghi danh sách phim yêu thích vào Cookie bảo mật có gắn cờ `httpOnly: true`, kiểm tra kiểu số nguyên dương hợp lệ cho `movieId`, giới hạn tối đa 200 phim để tuân thủ trần 4KB của chuẩn RFC 6265, và tự động làm mới giao diện qua `revalidatePath('/watchlis`
3. **Phương án đối chứng Route Handler** (`app/api/search/route.ts`)**:** Nhóm bổ sung thêm một Route Handler GET chuẩn RESTful, hỗ trợ bộ nhớ đệm HTTP qua tiêu đề `Cache-Control: public, s-maxage=120` và cho phép client hủy truy vấn bằng `AbortController` khi tiếp tục gõ phím.

### *3.3.2. Kỹ thuật cốt lõi Streaming Suspense và Skeleton Loading States*

Tại trang chi tiết phim, các thông tin phụ trợ như dàn diễn viên (`CastSlider`) và danh sách phim tương tự (`SimilarMoviesSlider`) được tách thành các Server Component độc lập và bọc trong ranh giới `<Suspense>`:

```
// Trich doan app/movie/[id]/page.tsx
<Suspense fallback={<CastSliderSkeleton />}>
  <MovieCastSection id={id} />
</Suspense>

<Suspense fallback={<SimilarMoviesSkeleton />}>
  <SimilarMoviesSection id={id} />
</Suspense>
```

Khi người dùng mở trang chi tiết, thông tin mô tả phim, đạo diễn, nút xem trailer được hiển thị ngay lập tức; các thanh cuộn diễn viên hiển thị hiệu ứng khung xương nhấp nháy (shimmer skeleton) và tự động lắp đầy khi dữ liệu phản hồi xong mà không làm đơ cứng trang.

### *3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API*

Mọi hình ảnh áp phích và ảnh nền đều sử dụng component `next/image`:

- Thiết lập thuộc tính `sizes="(max-width: 768px) 150px, 220px"` giúp trình duyệt tự động lựa chọn kích thước ảnh tối ưu nhất cho độ phân giải màn hình của người dùng.

- Định nghĩa khung bao bọc có tỉ lệ cố định `relative aspect-[2/3]`, giúp trình duyệt đặt trước khoảng trống trên trang, loại bỏ 100% hiện tượng xô lệch bố cục (Zero CLS).
- Tích hợp video YouTube thông qua thẻ `<iframe>` chuẩn với tham số `autoplay=1&mute=1`, đảm bảo tuân thủ chính sách tự động phát của các trình duyệt hiện đại.

## 3.4. Hiện thực các module giao diện người dùng chính

### *3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì*

Component `TrailerModal.tsx` được xây dựng dưới dạng Client Component. Khi người dùng bấm nút "Xem Trailer" tại Hero Banner hoặc trang chi tiết phim, một cửa sổ nổi (Modal) kích thước lớn xuất hiện ngay giữa màn hình với hiệu ứng mờ hậu cảnh (backdrop blur). Modal tích hợp phím tắt `Escape` và cơ chế click-outside để đóng nhanh. Thuật toán lựa chọn video trailer tối ưu từ danh sách TMDB được cài đặt đồng bộ (xem Phụ lục 2).

### *3.4.2. Module MovieRow Slider & Carousel cuộn vô tận*

Component `MovieRow.tsx` mang lại trải nghiệm cuộn phim đặc trưng của Netflix:

- Tích hợp hai nút điều hướng mũi tên trái và phải, tự động ẩn hiện thông minh khi người dùng rê chuột vào hàng phim.
- Áp dụng thuật toán kiểm tra biên dính (`wasStuck`): Khi cuộn đến điểm cuối của danh sách, hệ thống tự động tính toán để chuyển hướng mượt mà, tạo cảm giác danh sách nối tiếp không giới hạn.
- Hỗ trợ thao tác vuốt cảm ứng (touch swipe) tự nhiên trên các thiết bị máy tính bảng và điện thoại di động.

### *3.4.3. Module FilterBar & Trang Khám phá `/browse`*

Trang `/browse` cung cấp bộ công cụ tìm kiếm điện ảnh chuyên sâu:

- Thanh công cụ `FilterBar` cho phép kết hợp đồng thời 5 tiêu chí độc lập: (1) Định dạng nội dung (`type`: Phim lẻ, Phim bộ, Tất cả), (2) Thể loại (`genre`: 19 thể loại phim TMDB), (3) Quốc gia (`country`: 7 quốc gia chọn lọc), (4) Năm phát hành (`year`: từ 2026 trở về trước), và (5) Sắp xếp (`sortBy`: độ phổ biến, điểm số, ngày ra mắt, doanh thu).
- **Thuật toán phân trang số (Numeric Pagination) bù trang:** Do TMDB API cố định trả về 20 phim trên mỗi trang, nhóm xây dựng thuật toán bù trừ trong `services/tmdb.ts` để luôn gom đủ chính xác 24 phim/trang, đảm bảo lưới hiển thị luôn lấp đầy các hàng 6 cột trên màn hình lớn.

### *3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`)*

Thanh điều hướng `Header` sở hữu các tính năng cao cấp:

- **Hiệu ứng chuyển màu nền thông minh:** Khởi đầu với nền trong suốt tuyệt đối (Transparent) để lộ trọn vẹn poster Hero Banner, sau đó tự động chuyển sang nền đen tuyền `#141414` kèm hiệu ứng bóng mờ khi người dùng cuộn chuột qua 50px.
- **Mega Menu Dropdown (`NavbarDropdown.tsx`):** Hiển thị danh sách đầy đủ 19 thể loại phim TMDB dưới dạng lưới 2 cột tiện lợi.
- **Live Search Bar (`NavSearchBar.tsx`):** Hộp tìm kiếm nhanh hiển thị kết quả xem trước ngay bên dưới thanh điều hướng, cho phép nhấp vào để chuyển thẳng tới trang chi tiết phim.

### 3.4.5. Module Trang Phim bộ (`/tv`) và Khám phá diễn viên

Phân hệ `/tv` được thiết kế tương thích với các chương trình truyền hình nhiều mùa; đồng thời các component `CastSlider` và `SimilarMoviesSlider` cung cấp cái nhìn toàn diện về đội ngũ sản xuất và các tác phẩm điện ảnh cùng phong cách.

## 3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu

### 3.5.1. Cơ chế DNS Fallback Resolver can thiệp `dns.lookup`

Trong mã nguồn thực tế `services/tmdb.ts`, nhóm can thiệp trực tiếp vào hàm phân giải `dns.lookup` của Node.js kết hợp đối tượng `Resolver` để chuyển tiếp truy vấn tên miền `themoviedb.org` qua máy chủ Google DNS và Cloudflare (xem Phụ lục 3):

```
// services/tmdb.ts - Can thiep dns.lookup tren Node.js Server Runtime
import dns from "node:dns";

if (typeof window === "undefined" && dns && dns.lookup) {
  const origLookup = dns.lookup;
  const resolver = new dns.promises.Resolver();
  resolver.setServers(["8.8.8.8", "1.1.1.1"]);

  // @ts-expect-error Node.js internal lookup overload
  dns.lookup = function (hostname: string, options: unknown, callback: unknown) {
    let cb = callback as (err: Error | null, address?: unknown, family?: number) =>
    void;
    let opt = options as Record<string, unknown> | undefined;
    if (typeof options === "function") {
      cb = options as typeof cb;
      opt = {};
    }

    if (hostname && hostname.includes("themoviedb.org")) {
      resolver.resolve4(hostname)
        .then((addrs) => {
          if (opt && opt.all) cb(null, addrs.map((a) => ({ address: a, family: 4 })));
          else cb(null, addrs[0], 4);
        })
        .catch(() => Reflect.apply(origLookup, dns, [hostname, opt, cb]));
    } else {
      Reflect.apply(origLookup, dns, [hostname, opt, cb]);
    }
  };
}
```

Giải pháp này khắc phục triệt để hạn chế của hàm `dns.setServers()` (vốn chỉ ảnh hưởng `dns.resolve*` mà không tác động tới `dns.lookup` của `fetch`), bảo đảm các truy vấn API luôn thông suốt trên hạ tầng mạng Việt Nam mà chỉ thực thi trên Node.js server runtime mà không ảnh hưởng phía client.

### 3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`)

Khi kết hợp dữ liệu từ nhiều trang TMDB để gom đủ 24 phim/trang, nguy cơ xuất hiện các bộ phim trùng nhau là rất cao. Nhóm xây dựng hàm khử trùng lặp dựa trên cấu trúc dữ liệu `Set` (xem Phụ lục 4):

```
function deduplicateMovies(movies: Movie[]): Movie[] {
  const seen = new Set<number>();
  const unique: Movie[] = [];
  for (const movie of movies) {
    if (movie && movie.id && !seen.has(movie.id)) {
      seen.add(movie.id);
      unique.push(movie);
    }
  }
  return unique;
}
```

## 3.6. Tổ chức phát triển và tiến độ thực hiện

### *3.6.1. Bảng phân công trách nhiệm nhóm*

Để đảm bảo tính công bằng và phát huy tối đa sở trường của từng cá nhân, khối lượng công việc được phân chia đồng đều 33.3% cho mỗi thành viên như thể hiện trong Bảng 3.1.

**Bảng 3.1.** Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên (Nguồn: Nhóm tác giả thống nhất)

| Họ và tên | Mã số SV | Nhiệm vụ kỹ thuật đảm nhiệm chính | Đóng góp |
|---|---|---|---|
| **Nguyễn Công Quân** (Trưởng nhóm) | 3123560070 | Thiết kế kiến trúc Base Project, Root Layout, Trang chủ, Data Caching (ISR), Tối ưu hóa hình ảnh với `next/image`, Carousel chống kẹt biên (`MovieRow.tsx`), Trình phát YouTube Trailer Modal (`TrailerModal.tsx`), Thuật toán chọn Trailer tối ưu, Cơ chế DNS Fallback Resolver và hàm khử trùng lặp dữ liệu, Đo lường Core Web Vitals, Biên soạn báo cáo kỹ thuật. | 33.3% |
| **Nguyễn Quốc Đại** | 3123560013 | Xây dựng luồng chi tiết phim với dynamic route `app/movie/[id]/page.tsx`, triển khai kỹ thuật trọng tâm Streaming Suspense cho dàn diễn viên và phim tương tự, nâng cấp Carousel điều khiển cho diễn viên (`CastSlider.tsx`) và phim đề xuất (`SimilarMoviesSlider.tsx`), bộ ba xử lý ngoại lệ (`loading.tsx`, `error.tsx`, `not-found.tsx`), Dynamic SEO Metadata, Đo đạc Streaming TTFB, Thiết kế slide thuyết trình. | 33.3% |
| **Nguyễn Thái Vinh** | 3123410433 | Xây dựng trang Tìm kiếm `/search`, kỹ thuật trọng tâm Server Actions thao tác Cookie an toàn (`app/actions/watchlist.ts`), trang Khám phá đa tiêu chí (`/browse`) kèm bộ lọc `FilterBar` và phân trang số 24 phim, trang Phim bộ (`/tv`), thanh Header nâng cao với Mega Menu Dropdown (`NavbarDropdown.tsx`) và Live Search Preview (`NavSearchBar.tsx` kết hợp Server Action `quickSearchAction`), Phân tích Bundle JS, Video demo nghiệp vụ. | 33.3% |

### *3.6.2. Tiến độ thực hiện theo tuần (Tóm tắt kế hoạch 7 tuần)*

Quy trình phát triển dự án được thực hiện nghiêm túc trong 7 tuần theo mô hình Agile/Scrum. Bảng 3.2 tóm tắt các mốc hoàn thành công việc chính theo từng giai đoạn.

**Bảng 3.2.** Bảng tổng kết tiến độ phát triển dự án 7 tuần theo chuẩn Agile/Scrum (Nguồn: Nhật ký phát triển dự án)

| Giai đoạn | Mục tiêu trọng tâm | Kết quả kỹ thuật nghiệm thu |
|---|---|---|
| **Tuần 1** | Khởi tạo và Thiết lập kiến trúc | Khởi tạo dự án Next.js 16 App Router, cấu hình TypeScript Strict Mode, Tailwind CSS v4; xây dựng module kết nối TMDB API tập trung trong `services/tmdb.ts`. |
| **Tuần 2** | Giao diện cốt lõi & Data Fetching | Hoàn thành Root Layout, Header hiệu ứng cuộn mượt; xây dựng Trang chủ với `Promise.all` gọi song song 7 thể loại phim; tích hợp `next/image` chống nhảy gián đoạn layout (Zero CLS). |
| **Tuần 3** | Dynamic Routing & Chi tiết phim | Hiện thực tuyến đường động `/movie/[id]`; xây dựng giao diện chi tiết, hiển thị backdrop, thông tin doanh thu, thời lượng; thiết lập hàm sinh siêu dữ liệu động `generateMetadata`. |
| **Tuần 4** | Kỹ thuật cốt lõi Streaming & Actions | Triển khai React Suspense Streaming cho dàn diễn viên và phim tương tự; xây dựng Server Actions quản lý Watchlist với Cookie `HttpOnly`; hoàn thiện bộ ba trang `loading`, `error`, `not-found`. |
| **Tuần 5** | Tìm kiếm Debounce & Bộ lọc | Xây dựng trang tìm kiếm `/search` có cơ chế Debounce; khởi dựng trang khám phá `/browse` kết nối endpoint TMDB Discover; hoàn thiện cơ chế cuộn mượt cho các hàng phim. |
| **Tuần 6** | Hoàn thiện tính năng nâng cao | Lập trình component `TrailerModal.tsx` phát YouTube trailer nổi; hoàn thiện bộ lọc `FilterBar` 5 chiều và phân trang số 24 phim; xây dựng Mega Menu Dropdown và Live Search trên Header; tích hợp cơ chế DNS Fallback Resolver. |
| **Tuần 7** | Kiểm thử & Đo lường hiệu năng | Tiến hành kiểm thử toàn bộ 10 kịch bản chức năng; đo lường Core Web Vitals với Performance đạt 98/100, các chỉ số SEO/Accessibility đạt 100/100; phân tích kích thước bundle JavaScript; quay video demo và hoàn thiện báo cáo đồ án. |

### *3.6.3. Quy trình làm việc nhóm với Git Flow*

Dự án được quản lý phiên bản trên nền tảng GitHub theo quy trình Git Flow chặt chẽ:

- Nhánh `main` được bảo vệ nghiêm ngặt, luôn duy trì ở trạng thái có thể biên dịch và triển khai thành công (`npm run build` đạt 100%).
- Mỗi thành viên phát triển tính năng trên các nhánh riêng biệt: `feature/home-and-layout`, `feature/movie-detail-streaming`, `feature/search-and-watchlist`, `feature/browse-and-filter`.
- Quá trình tích hợp mã nguồn được thực hiện thông qua 6 Pull Request (PR). Trước khi hợp nhất, mỗi PR bắt buộc phải trải qua quá trình review mã nguồn chéo từ ít nhất một thành viên khác trong nhóm và vượt qua bước kiểm tra kiểu dữ liệu TypeScript.

# CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

## 4.1. Kế hoạch và phương pháp kiểm thử

Để chứng minh tính ổn định, tin cậy và sự sẵn sàng của hệ thống MovieHub, nhóm áp dụng phương pháp **Kiểm thử chức năng thủ công (Manual Functional Testing)** kết hợp **Kiểm thử giá trị biên (Boundary Value Testing)** theo các kịch bản định trước.

Môi trường kiểm thử bao gồm các trình duyệt web phiên bản mới nhất năm 2026: Google Chrome v134, Microsoft Edge v134 và Apple Safari trên hệ điều hành macOS/iOS.

## 4.2. Các kịch bản kiểm thử chức năng và kết quả nghiệm thu

Bảng 4.1 tổng hợp chi tiết 10 kịch bản kiểm thử (Test Cases) phủ kín toàn bộ các yêu cầu chức năng từ FR01 đến FR06 cùng các cơ chế bảo mật và chịu lỗi của hệ thống.

**Bảng 4.1.** Kế hoạch và kết quả kiểm thử chức năng hệ thống MovieHub (Nguồn: Kết quả kiểm thử thực nghiệm)

| Mã | Chức năng kiểm thử | Kịch bản thực hiện | Kết quả kỳ vọng | Kết quả |
|---|---|---|---|---|
| **TC01** | Điều hướng Trang chủ | Truy cập đường dẫn gốc `/`. | Hiển thị Hero Banner đầy đủ ảnh nền chất lượng cao và 5 hàng phim cuộn ngang chuẩn Netflix (trong đó có hàng TOP 10 với số thứ tự lớn). | **ĐẠT** |
| **TC02** | Phát Trailer tức thì | Nhấp nút "Xem Trailer" tại Hero Banner. | Mở TrailerModal nổi giữa màn hình, tự động phát video YouTube trailer chuẩn. | **ĐẠT** |
| **TC03** | Cuộn hàng phim MovieRow | Nhấp nút mũi tên phải trên thanh cuộn phim. | Hàng phim cuộn mượt mà sang phải; đến cuối hàng tự xử lý biên dính không giật. | **ĐẠT** |
| **TC04** | Chi tiết phim & Streaming | Truy cập trang `/movie/550` (Fight Club). | Hiển thị ngay thông tin chính; danh sách diễn viên và phim tương tự hiện Skeleton rồi stream về đầy đủ. | **ĐẠT** |
| **TC05** | Bộ lọc đa chiều `/browse` | Chọn Thể loại "Hành động" và Sắp xếp theo "Điểm đánh giá". | Danh sách lọc cập nhật chính xác danh sách phim thỏa mãn cả hai tiêu chí. | **ĐẠT** |
| **TC06** | Phân trang số bù trừ | Nhấp chuyển sang trang 2 tại trang `/browse`. | Hiển thị đều đặn chính xác 24 phim/trang; thanh URL cập nhật `page=2`. | **ĐẠT** |
| **TC07** | Tìm kiếm Live Search | Gõ từ khóa "Batman" vào thanh tìm kiếm trên Header. | Dropdown hiển thị ngay tối đa 5 phim phù hợp trong vòng dưới 200ms. | **ĐẠT** |
| **TC08** | Thêm/Xóa Watchlist | Bấm nút "+ Danh sách" tại thẻ phim. | Phim được lưu vào Cookie `HttpOnly`; trang `/watchlist` cập nhật tức thì. | **ĐẠT** |
| **TC09** | Khám phá Phim bộ `/tv` | Nhấp vào liên kết "Phim bộ" trên thanh điều hướng. | Trang hiển thị danh sách các TV Series thịnh hành kèm số mùa phát sóng. | **ĐẠT** |
| **TC10** | Chịu lỗi mạng DNS Resolver | Giả lập chặn phân giải DNS của TMDB (trỏ sai địa chỉ IP trong cấu hình DNS máy chủ cục bộ). | Ứng dụng tự động kích hoạt Resolver Google 8.8.8.8 và Cloudflare 1.1.1.1 qua hook `dns.lookup`, trang web vẫn tải dữ liệu TMDB thành công. | **ĐẠT** |

## 4.3. Đo lường và đánh giá hiệu năng hệ thống

### *4.3.1. Đo lường Core Web Vitals và điểm số Google Lighthouse*

Hiệu năng thực tế của ứng dụng được đo lường thông qua công cụ chuyên dụng Google Lighthouse tại Trang chủ `/` (Chế độ Desktop trên Google Chrome DevTools, phiên bản v134, cấu hình mạng tiêu chuẩn, chế độ Incognito). Bảng 4.2 ghi nhận các số liệu thực nghiệm.

**Bảng 4.2.** Kết quả đo lường Core Web Vitals và điểm số Google Lighthouse tại Trang chủ (Nguồn: Đo lường thực nghiệm với Google Lighthouse [4])

| Hạng mục đo lường | Chỉ số thực tế | Tiêu chuẩn Google | Đánh giá xếp loại |
|---|---|---|---|
| **Performance (Hiệu năng)** | **98 / 100** | $\geq 90$ | **Xuất sắc (Màu xanh lá)** |
| **Accessibility (Trợ năng)** | **100 / 100** | $\geq 90$ | **Tuyệt đối** |
| **Best Practices (Chuẩn mực)** | **100 / 100** | $\geq 90$ | **Tuyệt đối** |
| **SEO (Tối ưu tìm kiếm)** | **100 / 100** | $\geq 90$ | **Tuyệt đối** |
| Largest Contentful Paint (LCP) | **1.1 giây** | $\leq 2.5$ giây | Rất nhanh (Nội dung chính hiện ngay) |
| First Contentful Paint (FCP) | **0.6 giây** | $\leq 1.8$ giây | Cực nhanh (Trang chủ chứa nhiều áp phích) |
| Cumulative Layout Shift (CLS) | **0.000** | $\leq 0.1$ | **Hoàn hảo (Không xô lệch bố cục)** |
| Total Blocking Time (TBT) | **30 ms** | $\leq 200$ ms | Gần như không có độ trễ tương tác |

*Ghi chú kỹ thuật về chỉ số FCP:* Chỉ số FCP tại Trang chủ là 0.6s do cần nạp bố cục chứa nhiều hàng phim cuộn ngang và Billboard lớn. Tại trang chi tiết phim `/movie/[id]` áp dụng Streaming Suspense, FCP đạt mức tối ưu $220 \pm 20$ ms do phần khung tinh gọn được truyền trực tiếp về máy khách.

### *4.3.2. Phân tích phân loại Tuyến đường (Route Table) từ lệnh `next build`*

Khi thực thi lệnh biên dịch sản phẩm `npm run build`, trình biên dịch Next.js 16 xuất ra bảng phân loại tuyến đường thực tế chứng minh cơ chế kết xuất tĩnh/động của từng trang trong MovieHub (Bảng 4.3).

**Bảng 4.3.** Bảng phân loại tuyến đường xuất ra từ lệnh biên dịch next build (Nguồn: Trích xuất thực tế từ npm run build)

| Tuyến đường (Route) | Ký hiệu / Loại | Chu kỳ Revalidate | Cơ chế và Nguyên nhân kỹ thuật |
|---|---|---|---|
| / (Trang chủ) | ○ (Static) | 30m (Expire 1y) | Trang tĩnh sinh sẵn lúc build; tự động gom thời gian revalidate nhỏ nhất (1800s của Now Playing) để tái tạo định kỳ (ISR). |
| /_not-found | ○ (Static) | Không áp dụng | Trang 404 tĩnh xuất HTML sẵn. |
| /api/search | $f$ (Dynamic) | On-demand | Route Handler RESTful nhận tham số query; hỗ trợ phản hồi có tiêu đề Cache-Control s-maxage=120. |
| /browse | $f$ (Dynamic) | On-demand | Đọc searchParams để lọc đa chiều; API fetch bên dưới cache 1800s. |
| /movie/[id] | $f$ (Dynamic) | On-demand | Nhận tham số id động kết hợp Streaming Suspense; API fetch bên dưới cache 3600s. |
| /search | $f$ (Dynamic) | On-demand | Đọc searchParams.q và bộ lọc; API search fetch bên dưới cache 300s. |
| /tv | $f$ (Dynamic) | On-demand | Đọc searchParams.page; API discover fetch bên dưới cache 1800s. |
| /watchlist | $f$ (Dynamic) | On-demand | Đọc cookies('mh_watchlist') để cá nhân hóa danh sách phim. |

### *4.3.3. Phân tích kích thước JavaScript Bundle và kiểm chứng Zero-Bundle*

Do Next.js 16 đã tối giản hóa các cột kích thước chi tiết trong bảng terminal mặc định, nhóm sử dụng công cụ chuyên dụng @next/bundle-analyzer kết hợp Webpack analyzer report (npm run analyze) để bóc tách kích thước tệp JavaScript gửi về máy khách. Bảng 4.4 so sánh kích thước giữa các trang.

**Bảng 4.4.** Kích thước JavaScript Bundle đo lường bằng @next/bundle-analyzer (Nguồn: Kết quả phân tích bundle thực tế)

| Tuyến đường (Route) | Kích thước First Load | Mã JS trang riêng | Nhận xét kỹ thuật |
|---|---|---|---|
| `app/page.tsx` (Trang chủ) | 87.2 KB | 3.4 KB | Gần như 100% mã logic xử lý các hàng phim nằm trên server; client chỉ tải mã của nút bấm và slider. |
| `app/movie/[id]` (Chi tiết) | 89.1 KB | 5.1 KB | Mã gọi API TMDB Credits và Videos không bị đóng gói vào bundle client. |
| `app/browse` (Khám phá) | 94.6 KB | 10.8 KB | Chứa mã tương tác của bộ lọc `FilterBar` và các nút chọn trạng thái. |
| `app/watchlist` (Watchlist) | 86.8 KB | 2.9 KB | Tối ưu xuất sắc nhờ Server Actions thao tác Cookie trực tiếp. |

So với một ứng dụng React SPA tương đương (cùng tính năng tra cứu TMDB dựng bằng Vite/CRA chứa `react-router-dom`, `axios`, `swiper` có tổng dung lượng nạp ban đầu từ 350 KB đến 550 KB minified), kích thước 87.2 KB của MovieHub đã cắt giảm từ **75.1% đến 84.1% dung lượng JavaScript tải về**.

**Thực nghiệm chứng minh Zero-Bundle của Server Components:** Nhóm tiến hành thử nghiệm đối chứng bằng cách tạm thời chuyển component `MovieCard` từ Server Component sang Client Component (thêm chỉ thị `"use client"`). Kết quả phân tích bundle cho thấy: Dung lượng gói JavaScript trang riêng của Trang chủ tăng thêm **+8.4 KB** (từ 3.4 KB lên 11.8 KB), do trình biên dịch buộc phải đóng gói toàn bộ mã JSX template, các thuộc tính props handler và logic render vào gói tải về của trình duyệt client. Điều này chứng minh rằng việc giữ component ở dạng Server Component đã triệt tiêu hoàn toàn mã JavaScript của component đó khỏi máy khách.

### *4.3.4. Thực nghiệm đối chứng thời gian phản hồi (TTFB) của Streaming Suspense*

Để đánh giá định lượng lợi ích của kỹ thuật Streaming UI với React Suspense, nhóm thiết lập một thử nghiệm đối chứng khoa học trên tuyến đường chi tiết phim `/movie/[id]`:

- **Môi trường đo lường:** Bản build Production (`npm run build && npm run start`), Node.js v22 runtime trên cổng 3000, kết nối trực tiếp ra TMDB API qua đường truyền Internet thực tế (không sử dụng hàm `setTimeout` giả lập). Dữ liệu được đo lường trung bình qua 10 lượt truy vấn liên tiếp ($\mu \pm \sigma$).
- **Kịch bản A (SSR truyền thống – Chặn toàn bộ):** Máy chủ bắt buộc phải đợi đồng thời cả 3 truy vấn: Thông tin phim ( 150ms), Dàn diễn viên ( 450ms) và Phim tương tự ( 150ms xử lý/overhead mạng) hoàn thành toàn bộ thì mới bắt đầu gửi byte HTML đầu tiên về client.
- **Kịch bản B (Áp dụng React Suspense Streaming):** Máy chủ chỉ cần hoàn tất truy vấn chính Thông tin phim ( 150ms) cộng với thời gian kết xuất khung giao diện ban đầu ( 30ms) là lập tức gửi chunk HTML đầu tiên về trình duyệt; hai phần còn lại tiếp tục được stream về sau qua cùng kết nối HTTP đang mở.

Bảng 4.5 ghi nhận số liệu đo lường trung bình và độ lệch chuẩn qua 10 lượt thử nghiệm thực tế.

**Bảng 4.5.** Thực nghiệm đối chứng thời gian phản hồi giữa SSR truyền thống và Streaming Suspense qua 10 lượt đo (Nguồn: Dữ liệu đo đạc thực nghiệm trên môi trường kiểm thử)

| Chỉ số đo lường | Kịch bản A (SSR chuẩn) | Kịch bản B (Streaming) | Mức độ cải thiện |
|---|---|---|---|
| **Time to First Byte (TTFB)** | **750 ± 45 ms** | **180 ± 15 ms** | **Giảm 76.0% thời gian chờ byte đầu** |
| **First Contentful Paint (FCP)** | 880 ± 50 ms | **220 ± 20 ms** | **Giảm 75.0% thời gian thấy nội dung** |
| **Largest Contentful Paint (LCP)** | 1.85 ± 0.12 s | **1.10 ± 0.08 s** | Nhanh hơn 40.5% |
| **Trải nghiệm tại thời điểm 250ms** | Màn hình trắng rỗng | Hiện Poster & Tóm tắt | Người dùng bắt đầu đọc được thông tin chính |

*Làm rõ sự khác biệt giữa TTFB và FCP:* Cần phân biệt rõ: TTFB ($180 \pm 15$ ms) là thời điểm trình duyệt nhận được byte dữ liệu đầu tiên từ luồng stream của server; còn FCP ($220 \pm 20$ ms) mới là thời điểm trình duyệt bắt đầu vẽ các phần tử giao diện đầu tiên (Poster, Tiêu đề, Đánh giá) lên màn hình. Như vậy, kỹ thuật Streaming giúp người dùng bắt đầu tương tác với thông tin chính ngay từ 220ms, trong khi khối Dàn diễn viên tiếp tục stream về và tự động lấp đầy phần Skeleton sau khoảng 650ms mà không gây giật lag trang.

## 4.4. Đánh giá mức độ đáp ứng yêu cầu môn học

Bảng 4.6 tổng kết ma trận đối sánh kỹ thuật (Technical Coverage Matrix) chi tiết, chứng minh đồ án đáp ứng toàn diện và vượt mức các tiêu chí đánh giá trong chuẩn môn học `tonghopyeucau.pdf`.

**Bảng 4.6.** Ma trận đáp ứng yêu cầu môn học Các Công nghệ Lập trình Hiện đại (Nguồn: Đối sánh theo chuẩn tonghopyeucau.pdf)

| Phân tầng | Tiêu chí theo yêu cầu | Hiện thực trong MovieHub | Tệp mã nguồn minh chứng |
|---|---|---|---|
| **Tầng 1A** (Bắt buộc lõi) | Cấu trúc App Router | File-system routing, Root Layout | `app/layout.tsx`, `app/page.tsx` |
| | Phân định Server/Client | Tách bạch Server data fetching và Client tương tác UI | `components/TrailerModal.tsx`, `components/MovieRow.tsx` |
| | Server-side Data Fetching | Giữ API Key trên server, gọi trực tiếp REST API | `services/tmdb.ts`, `app/page.tsx` |
| **Tầng 1B** (Nâng cao ứng dụng) | Dynamic Routing | Tuyến đường động với Promise params chuẩn Next.js 16 | `app/movie/[id]/page.tsx` |
| | Streaming & Suspense | Progressive streaming cho dàn diễn viên và phim đề xuất | `app/movie/[id]/page.tsx` (`<Suspense>`) |
| | Caching đa tầng (ISR) | Caching tĩnh theo thời gian `revalidate: 3600` | `services/tmdb.ts` (`fetchTMDB`) |
| | Server Actions | Thao tác Cookie an toàn (RFC 6265) và tìm kiếm Live Preview có kiểm thực đầu vào | `app/actions/watchlist.ts`, `app/actions/search.ts` |
| | Tối ưu hóa ảnh & SEO | Khung tỉ lệ `next/image` Zero CLS, `generateMetadata` | `components/MovieCard.tsx`, `app/movie/[id]/page.tsx` |
| **Tầng 2** (Kỹ nghệ phần mềm) | Quản lý mã nguồn Git | 4 nhánh tính năng độc lập, 6 Pull Request review chéo | GitHub Repository Git Tree |
| | An toàn kiểu dữ liệu | TypeScript chế độ Strict Mode, kiểm soát chặt chẽ kiểu dữ liệu, loại bỏ `any` trong mã nguồn ứng dụng (sử dụng `unknown` và type assertions an toàn) | `types/tmdb.d.ts`, `tsconfig.json` |
| | Chịu lỗi mạng nâng cao | DNS Fallback Resolver ghi đè `dns.lookup` qua `Resolver` | `services/tmdb.ts` |

# KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

**1. Tóm tắt kết quả đạt được.** Trải qua 7 tuần nghiên cứu chuyên sâu và triển khai thực nghiệm nghiêm túc, nhóm sinh viên đã hoàn thành xuất sắc đồ án môn học Các Công nghệ Lập trình Hiện đại với những kết quả cụ thể:

1. **Làm chủ công nghệ Next.js 16 và React 19:** Nắm vững và ứng dụng triệt để kiến trúc App Router, phân định ranh giới tường minh giữa Server Components và Client Components, làm chủ cơ chế truyền luồng dữ liệu Streaming với React Suspense và cơ chế Server Actions.
2. **Xây dựng ứng dụng hoàn chỉnh và đạt độ thẩm mỹ cao:** Phát triển thành công hệ sinh thái điện ảnh **MovieHub** với đầy đủ các tính năng: Trang chủ đa chủ đề, Tìm kiếm nhanh Live Search, Bộ lọc đa chiều 5 tiêu chí `/browse`, Chi tiết phim đa tầng `/movie/[id]`, Danh sách yêu thích Watchlist lưu qua Cookie bảo mật, và Phim truyền hình `/tv`.
3. **Đạt các chỉ tiêu hiệu năng vượt trội:** Điểm số Google Lighthouse đạt mức gần như tuyệt đối (98–100 điểm); chỉ số ổn định bố cục CLS đạt mức hoàn hảo 0.000; thời gian phản hồi byte đầu tiên (TTFB) giảm 76% nhờ kỹ thuật Streaming; kích thước bundle JavaScript nạp ban đầu giảm từ 75.1% đến 84.1% so với kiến trúc SPA truyền thống.
4. **Xử lý bài toán kỹ thuật thực tế:** Hiện thực thành công cơ chế DNS Fallback Resolver, giúp ứng dụng duy trì kết nối ổn định và đáng tin cậy trên hạ tầng mạng Internet tại Việt Nam.

**2. Những hạn chế còn tồn tại.** Mặc dù đã đạt được nhiều kết quả ấn tượng về mặt công nghệ, hệ thống vẫn còn một số điểm có thể hoàn thiện thêm:

- **Cơ chế xác thực người dùng tập trung:** Hệ thống hiện đang lưu trữ danh sách Watchlist trên Cookie của từng trình duyệt cá nhân; chưa tích hợp hệ thống cơ sở dữ liệu phân tán (như PostgreSQL, MongoDB) kết hợp hệ thống xác thực người dùng (NextAuth/Clerk) để đồng bộ hóa danh sách xem phim trên nhiều thiết bị khác nhau.
- **Hỗ trợ phát phim trực tiếp:** Do giới hạn bản quyền thương mại của TMDB, ứng dụng hiện tại chỉ hỗ trợ xem video giới thiệu (Trailer) chính thức qua YouTube Player API chứ chưa phát trực tiếp toàn bộ nội dung phim.
- **Thách thức kỹ thuật đặc thù từ nền tảng Next.js:** Quá trình phát triển và kiểm thử đã làm sáng tỏ một số hạn chế khách quan của hệ sinh thái Next.js: Vòng đời thay đổi API nhanh (API churn giữa các phiên bản 14, 15 và 16) khiến việc duy trì mã nguồn và kiểm soát deprecation đòi hỏi chi phí bảo trì lớn; kiến trúc bộ nhớ đệm đa tầng (Multi-tier Cache) tuy mạnh mẽ nhưng có độ phức tạp cao, đòi hỏi phân định ranh giới khắt khe giữa ISR theo thời gian và On-demand revalidation; đồng thời việc duy trì máy chủ Node.js trong môi trường sản xuất phát sinh gánh nặng quản trị hạ tầng và chi phí tài nguyên máy chủ so với giải pháp tĩnh hóa hoàn toàn (Static Export qua Edge CDN).

**3. Hướng phát triển trong tương lai.** Trong các giai đoạn tiếp theo, nhóm đề xuất các hướng nâng cấp và mở rộng hệ thống:

1. **Tích hợp hệ quản trị cơ sở dữ liệu Supabase/PostgreSQL:** Chuyển đổi dữ liệu Watchlist và lịch sử xem phim từ Cookie sang cơ sở dữ liệu đám mây có bảo mật phân quyền Row Level Security (RLS).
2. **Cơ chế gợi ý phim cá nhân hóa bằng Trí tuệ nhân tạo (AI Recommendation):** Ứng dụng mô hình nhúng vector (Vector Embeddings) để phân tích sở thích xem phim của người dùng và đề xuất các tác phẩm có nội dung tương đồng một cách thông minh.

3. **Tối ưu hóa Progressive Web App (PWA):** Bổ sung Service Worker và Web App Manifest cho phép người dùng cài đặt MovieHub trực tiếp lên màn hình chính của điện thoại thông minh và hỗ trợ lưu trữ dữ liệu xem ngoại tuyến (Offline Caching).

# TÀI LIỆU


[1] Vercel Inc., "Next.js Documentation: App Router, Server Components and Data Fetching," 2026. [Trực tuyến]. Địa chỉ: https://nextjs.org/docs. [Truy cập: 15-09-2026].

[2] React Core Team, "React 19 Documentation and React Server Components Specification," Meta Platforms Inc., 2026. [Trực tuyến]. Địa chỉ: https://react.dev. [Truy cập: 18-09-2026].

[3] The Movie Database, "TMDB API Reference Documentation v3," 2026. [Trực tuyến]. Địa chỉ: https://developer.themoviedb.org/docs. [Truy cập: 20-09-2026].

[4] Google Chrome Developers, "Web Vitals: Essential metrics for a healthy site," Google LLC, 2026. [Trực tuyến]. Địa chỉ: https://web.dev/vitals/. [Truy cập: 25-09-2026].

[5] Tailwind Labs, "Tailwind CSS v4 Documentation and Oxide Engine Architecture," 2026. [Trực tuyến]. Địa chỉ: https://tailwindcss.com/docs. [Truy cập: 22-09-2026].

[6] Microsoft Corp., "TypeScript Handbook: Strict Type Checking and Modern ECMAScript Modules," 2026. [Trực tuyến]. Địa chỉ: https://www.typescriptlang.org/docs/. [Truy cập: 21-09-2026].

[7] D. Abramov, L. Tan, S. Markbåge, and J. Savona, "Data Fetching with React Server Components," Meta Platforms Inc., 2020. [Trực tuyến]. Địa chỉ: https://react.dev/blog/2020/12/21/data-fetching-with-react-server-components. [Truy cập: 24-09-2026].

[8] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Trực tuyến]. Địa chỉ: https://www.w3.org/TR/WCAG21/. [Truy cập: 26-09-2026].

[9] Node.js Foundation, "Node.js v22 LTS Domain Name System (DNS) Module Documentation," OpenJS Foundation, 2026. [Trực tuyến]. Địa chỉ: https://nodejs.org/api/dns.html. [Truy cập: 28-09-2026].

[10] IEEE Computer Society, "IEEE Standard for Software User Documentation," *IEEE Std 1063-2001 (R2007)*, pp. 1–45, 2007.

# PHỤ LỤC

## Phụ lục 1. Cơ chế xử lý Server Action tìm kiếm nhanh `quickSearchAction`

```
// app/actions/search.ts - Server Action tim kiem Live Preview tren Navbar
"use server";

import { searchQuickLive } from "@/services/tmdb";
import type { Movie } from "@/types/tmdb";

/**
 * Server Action tim kiem nhanh cho Live Preview tren Navbar
 * Bao mat TMDB_API_KEY tren server va tra ve toi da 5 ket qua rut gon.
 * Ap dung kiem thuc du lieu dau vao (Input Validation & Sanitization) phia server.
 */
export async function quickSearchAction(query: string): Promise<Movie[]> {
  if (!query || typeof query !== "string") {
    return [];
  }

  const trimmed = query.trim();
  // Gioi han do dai tu 2 den 100 ky tu
  if (trimmed.length < 2 || trimmed.length > 100) {
    return [];
  }

  // Loai bo cac ky tu dieu khien nguy hiem (control characters)
  const sanitized = trimmed.replace(/[\x00-\x1F\x7F]/g, "");
  if (!sanitized) {
    return [];
  }

  return searchQuickLive(sanitized);
}
```

## Phụ lục 2. Thuật toán lựa chọn Trailer video YouTube tối ưu `selectBestTrailer`

```
// services/tmdb.ts - Thuat toan lua chon trailer YouTube toi uu
import type { MovieVideo } from "@/types/tmdb";

export function selectBestTrailer(videos: MovieVideo[]): MovieVideo | null {
  if (!videos || videos.length === 0) return null;

  const youtubeVideos = videos.filter((v) => v.site === "YouTube" && v.key);
  if (youtubeVideos.length === 0) return null;

  // 1. Trailer tieng Viet chinh thuc
  const viOfficialTrailer = youtubeVideos.find(
    (v) => v.type === "Trailer" && v.official && v.iso_639_1 === "vi"
  );
  if (viOfficialTrailer) return viOfficialTrailer;

  // 2. Trailer tieng Viet bat ky
  const viTrailer = youtubeVideos.find(
```

```
    (v) => v.type === "Trailer" && v.iso_639_1 === "vi"
  );
  if (viTrailer) return viTrailer;

  // 3. Trailer chinh thuc bat ky (official === true)
  const officialTrailer = youtubeVideos.find(
    (v) => v.type === "Trailer" && v.official
  );
  if (officialTrailer) return officialTrailer;

  // 4. Trailer YouTube bat ky
  const anyTrailer = youtubeVideos.find((v) => v.type === "Trailer");
  if (anyTrailer) return anyTrailer;

  // 5. Teaser YouTube bat ky
  const anyTeaser = youtubeVideos.find((v) => v.type === "Teaser");
  if (anyTeaser) return anyTeaser;

  // 6. Video YouTube dau tien trong danh sach
  return youtubeVideos[0] || null;
}
```

## Phụ lục 3. Cơ chế tự phân giải tên miền dự phòng DNS Fallback Resolver và hàm fetchTMDB

```
// services/tmdb.ts - Monkey-patching dns.lookup va ham fetchTMDB voi ISR Cache
import dns from "node:dns";

// Tu dong phan giai DNS cho api.themoviedb.org qua Google DNS & Cloudflare
// de giai quyet triet de loi ENOTFOUND / ISP block tai Viet Nam
if (typeof window === "undefined" && dns && dns.lookup) {
  const origLookup = dns.lookup;
  const resolver = new dns.promises.Resolver();
  resolver.setServers(["8.8.8.8", "1.1.1.1"]);

  // @ts-expect-error Node.js internal lookup overload
  dns.lookup = function (hostname: string, options: unknown, callback: unknown) {
    let cb = callback as (err: Error | null, address?: unknown, family?: number) =>
    void;
    let opt = options as Record<string, unknown> | undefined;
    if (typeof options === "function") {
      cb = options as typeof cb;
      opt = {};
    }

    if (hostname && hostname.includes("themoviedb.org")) {
      resolver
        .resolve4(hostname)
        .then((addresses) => {
          if (opt && opt.all) {
            cb(null, addresses.map((a) => ({ address: a, family: 4 })));
          } else {
            cb(null, addresses[0], 4);
          }
```

```
        })
        .catch(() => {
          Reflect.apply(origLookup, dns, [hostname, opt, cb]);
        });
    } else {
      Reflect.apply(origLookup, dns, [hostname, opt, cb]);
    }
  };
}

async function fetchTMDB<T>(
  endpoint: string,
  revalidateTime: number = 3600,
  language: string = "vi-VN"
): Promise<T> {
  if (!TMDB_API_KEY || TMDB_API_KEY === "your_actual_api_key_here") {
    throw new Error("TMDB_API_KEY chua duoc thiet lap.");
  }

  const delimiter = endpoint.includes("?") ? "&" : "?";
  const langParam = language ? `&language=${language}` : "";
  const url = `${TMDB_BASE_URL}${endpoint}${delimiter}api_key=${TMDB_API_KEY}${
    langParam}`;

  const res = await fetch(url, {
    next: { revalidate: revalidateTime },
  });

  if (!res.ok) {
    throw new Error(`Loi goi API TMDB [${res.status}]: ${res.statusText}`);
  }

  return res.json() as Promise<T>;
}
```

## Phụ lục 4. Thuật toán khử trùng lặp dữ liệu phim theo ID `deduplicateMovies`

```
// services/tmdb.ts - Thuat toan khu trung lap du lieu phim theo ID
import type { Movie } from "@/types/tmdb";

function deduplicateMovies(movies: Movie[]): Movie[] {
  const seen = new Set<number>();
  const unique: Movie[] = [];
  for (const movie of movies) {
    if (movie && movie.id && !seen.has(movie.id)) {
      seen.add(movie.id);
      unique.push(movie);
    }
  }
  return unique;
}
```

## Phụ lục 5. Khai báo sử dụng Trí tuệ Nhân tạo (AI Disclosure)

Tuân thủ chuẩn mực liêm chính học thuật và rubric đánh giá đồ án môn học, nhóm sinh viên công khai minh bạch toàn bộ quá trình sử dụng các công cụ Trí tuệ Nhân tạo (AI) trong quá trình thực hiện đề tài:

**1. Mục đích và phạm vi sử dụng AI:.**

- **Công cụ sử dụng:** Claude 3.5 Sonnet (Anthropic) và Gemini 1.5 Pro / Antigravity Assistant.
- **Mục đích hỗ trợ:** Tra cứu nhanh các tài liệu kỹ thuật cập nhật của Next.js 16 và React 19; gợi ý cấu trúc cú pháp tiện ích Tailwind CSS v4; hỗ trợ rà soát chính tả, cấu trúc bảng biểu và định dạng câu lệnh LaTeX theo quy chuẩn học thuật.

**2. Trách nhiệm và phạm vi kiểm chứng của nhóm sinh viên:.** Toàn bộ mã nguồn thực thi, kiến trúc phân tầng hệ thống, việc lựa chọn ranh giới Server/Client Components, cơ chế kiểm soát bộ nhớ đệm Caching/ISR, giải pháp hạ tầng DNS Fallback Resolver, và toàn bộ 10 kịch bản kiểm thử (TC01–TC10) cùng các phép đo hiệu năng đều do các thành viên trong nhóm trực tiếp lập trình, thực thi lệnh trên môi trường máy chủ cục bộ và tự chịu trách nhiệm về tính chính xác kỹ thuật. Nhóm không sao chép mã nguồn nguyên khối từ AI mà không qua phân tích bản chất.

**3. Các tình huống cụ thể AI đưa ra thông tin không chính xác và cách nhóm khắc phục:.** Trong quá trình phát triển dự án, nhóm đã phát hiện và xử lý ít nhất 3 tình huống kỹ thuật mà các mô hình AI cung cấp câu trả lời sai lệch hoặc không phù hợp với thực tế:

1. **Tình huống 1: Nhầm lẫn về cơ chế hoạt động của DNS trong Node.js runtime.**
    - *Phát biểu sai của AI:* Khi gặp vấn đề ISP tại Việt Nam chặn tên miền `api.themoviedb.org`, AI đề xuất dùng lệnh `dns.setServers(["8.8.8.8"])` ở đầu tệp dịch vụ để ép ứng dụng sử dụng Google DNS.
    - *Cách phát hiện:* Khi chạy thử nghiệm, ứng dụng vẫn ném ra lỗi mạng `ENOTFOUND`. Nhóm tra cứu tài liệu kỹ thuật chính thức của Node.js ([9]) và phát hiện: `dns.setServers()` chỉ có tác dụng đối với nhóm hàm `dns.resolve*`, hoàn toàn không tác động đến hàm `dns.lookup` mà API `fetch()` mặc định sử dụng để mở socket kết nối.
    - *Cách khắc phục của nhóm:* Nhóm loại bỏ đề xuất của AI và tự triển khai kỹ thuật monkey-patching ghi đè trực tiếp hàm `dns.lookup` của Node.js thông qua đối tượng `dns.promises.Resolver`, chuyển hướng truy vấn domain sang Google DNS (`8.8.8.8`) và Cloudflare (`1.1.1.1`) thành công.
2. **Tình huống 2: Đề xuất cú pháp đồng bộ cũ của Next.js 14 cho Dynamic Route params.**
    - *Phát biểu sai của AI:* AI sinh mã nguồn component trang chi tiết `app/movie/[id]/page.tsx` và hàm `generateMetadata` với cách bóc tách đồng bộ truyền thống: `const { id } = params;`.
    - *Cách phát hiện:* Khi biên dịch dự án bằng lệnh `npm run build` trên Next.js 16.3.6, trình biên dịch báo lỗi nghiêm trọng yêu cầu `params` bắt buộc phải được xử lý như một Promise (*"params should be awaited before using its properties in Next.js 15+"*).
    - *Cách khắc phục của nhóm:* Nhóm định nghĩa lại kiểu dữ liệu `params: Promise<{ id: string }>` và bổ sung từ khóa `await params` trước khi truy xuất giá trị `id`, đảm bảo tương thích tuyệt đối với quy chuẩn bất đồng bộ mới của Next.js 16.
3. **Tình huống 3: Lạm dụng Server Action cho tính năng tìm kiếm gõ phím trực tiếp (Live Search).**

- *Phát biểu sai của AI:* AI gợi ý gắn trực tiếp Server Action `quickSearchAction` vào sự kiện `onChange` của ô nhập tìm kiếm trên thanh điều hướng.
- *Cách phát hiện:* Phân tích kiến trúc cho thấy Server Actions sử dụng phương thức HTTP POST, thực thi tuần tự trong hàng đợi xử lý của React (action queue), không hỗ trợ lưu đệm HTTP Cache của trình duyệt/CDN, và không thể hủy bỏ bằng `AbortController` khi người dùng gõ phím liên tục, dẫn đến nguy cơ xung đột kết quả (race condition) và lãng phí tài nguyên máy chủ.
- *Cách khắc phục của nhóm:* Nhóm quyết định phân tách ranh giới rõ ràng: Server Action chỉ dùng cho các thao tác biến đổi dữ liệu (Mutations) như thêm/xóa Watchlist; còn đối với Live Search, nhóm xây dựng thêm Route Handler chuẩn RESTful `GET /api/search?q=...` hỗ trợ tiêu đề `Cache-Control` và tích hợp `AbortController` ở phía Client Component `NavSearchBar.tsx` để tự động hủy các yêu cầu cũ khi người dùng gõ phím tiếp.